UNIVERSITÄT
LEIPZIG

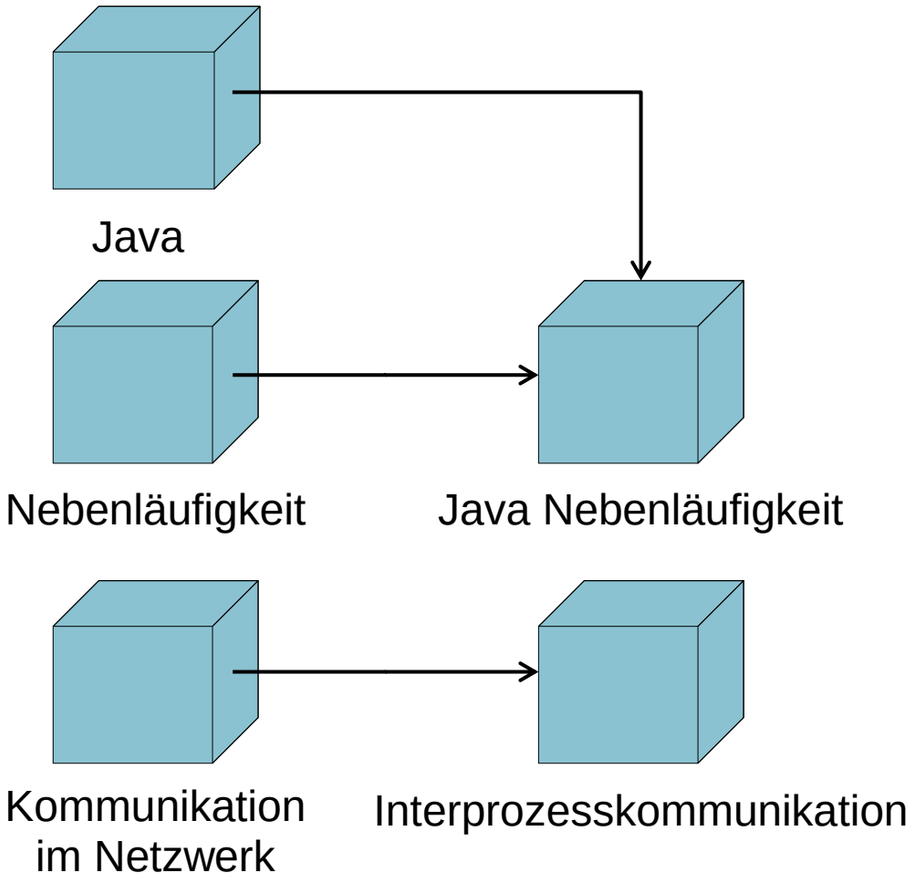
Interprozesskommunikation in Java

Entwicklung verteilter Anwendungen

Leipzig, 02.06.2022

Dr. Pascal Kovacs

AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Kommunikation im Netzwerk
Interprozesskommunikation

ZIELE INTERPROZESSKOMMUNIKATION

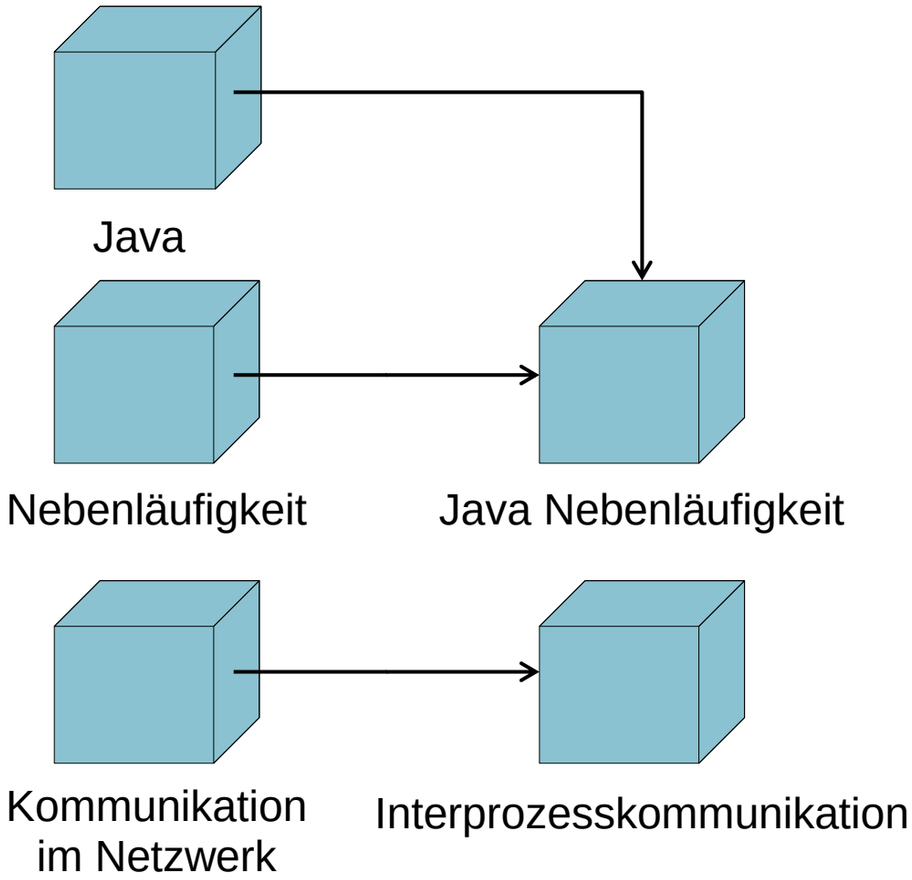
- IPK und Methoden verstehen
- IP und Ports
- TCP
- UDP
- Theorie Sockets

KONTROLLFRAGEN INTERPROZESSKOMMUNIKATION

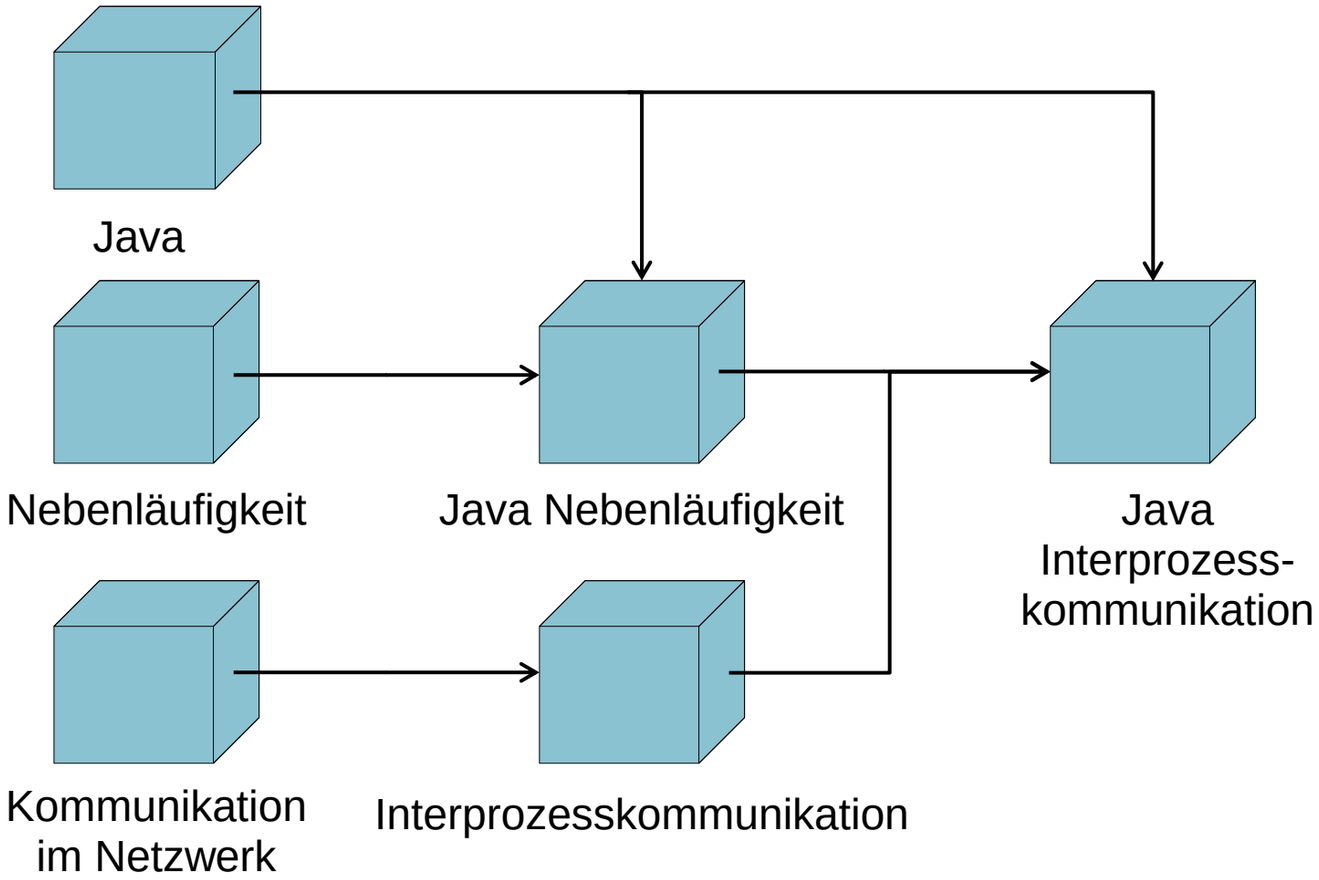
- Welche Methoden zur IPK gibt es?
- Wie hängen Netzwerkkommunikation und Interprozesskommunikation zusammen?
- Was bestimmen IP-Adressen?
- Was bestimmen Ports?
- Was ist der zentrale Unterschied zwischen TCP und UDP?
- Welche Eigenschaften ergeben sich daraus für TCP und UDP?
- Zu welchen Schichten des ISO/OSI Referenzmodells gehören IP, TCP und UDP?
- Wie viele Sockets braucht es für ein- und ausgehende IPK von zwei Teilnehmern?
- Wie funktionieren TCP und UDP mittels Sockets?

BEISPIELFRAGE

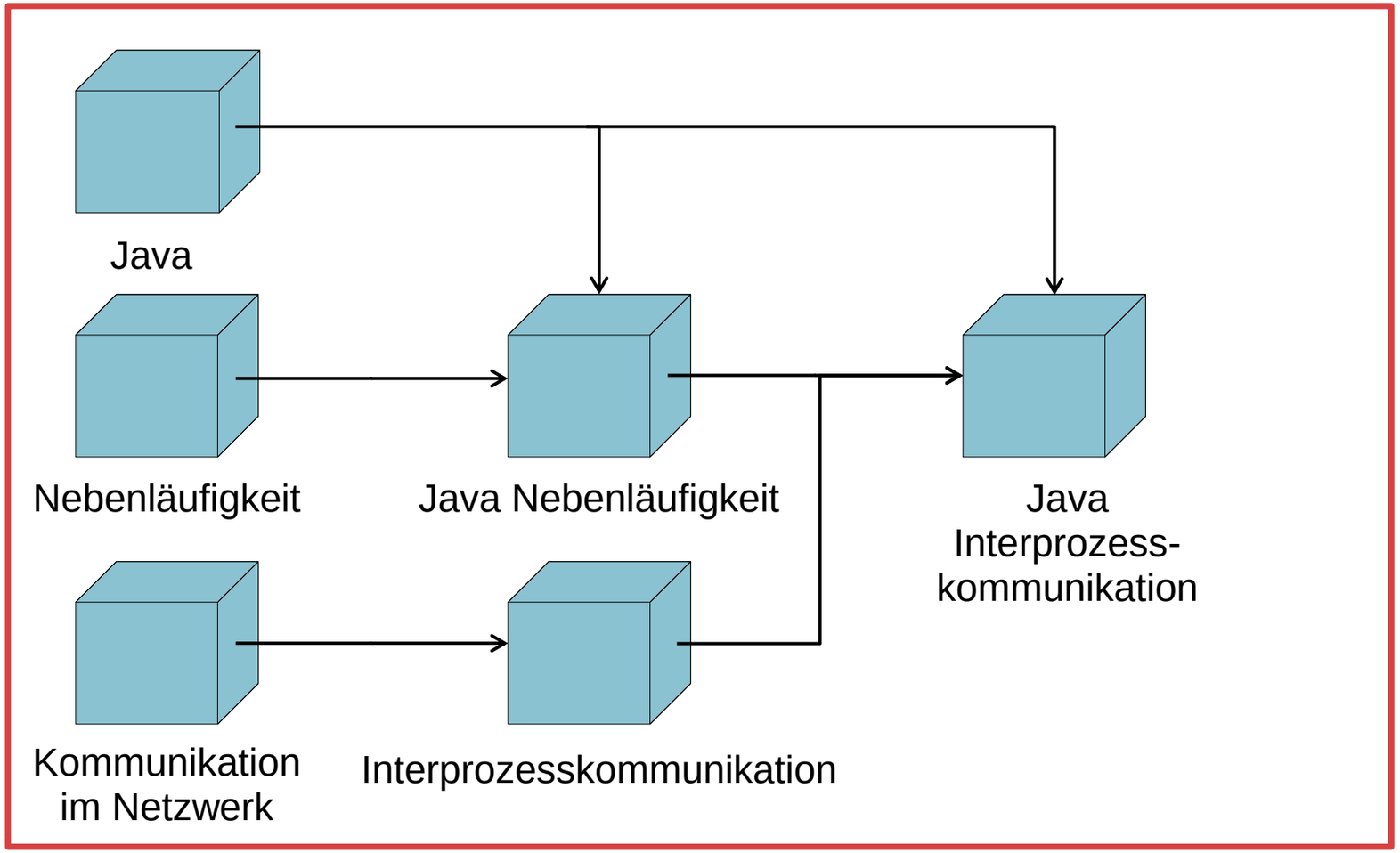
AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Kommunikation im Netzwerk
Interprozesskommunikation

AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozess-kommunikation
Kommunikation im Netzwerk
Interprozesskommunikation

AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozess-kommunikation
Kommunikation im Netzwerk
Interprozesskommunikation

ZIELE JAVA INTERPROZESSKOMMUNIKATION

- Sockets in Java verstehen und Code lesen können
- Java RMI verstehen und die Unterschiede zu Sockets kennen

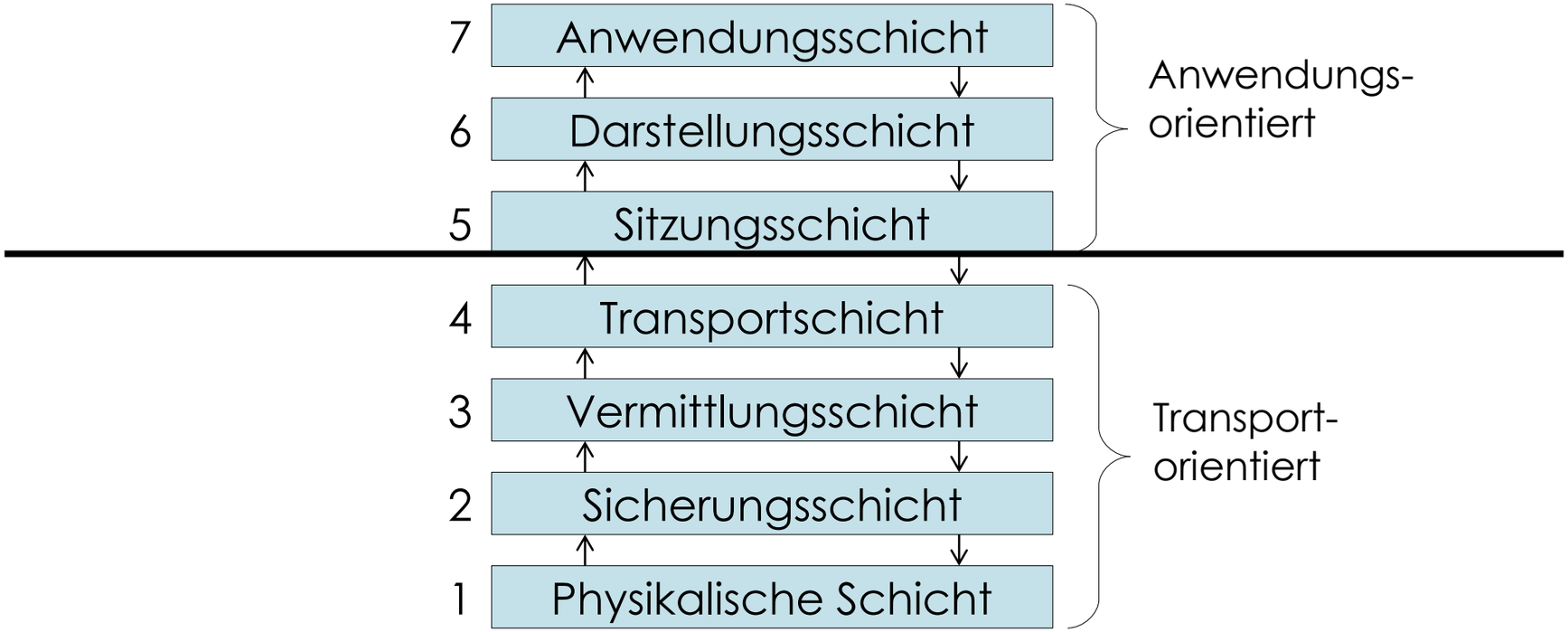
KONTROLLFRAGEN JAVA INTERPROZESSKOMMUNIKATION

- Welche Unterschiede gibt es bei der Implementierung einer IPK über TCP und UDP Sockets in Java?
- Welche Vorteile hat RMI im Vergleich zur IPK mittels Sockets?
- Wie ist die Architektur von RMI?
- Was ist der Unterschied zwischen Vererbung und dem Implementieren von Interfaces in Java?

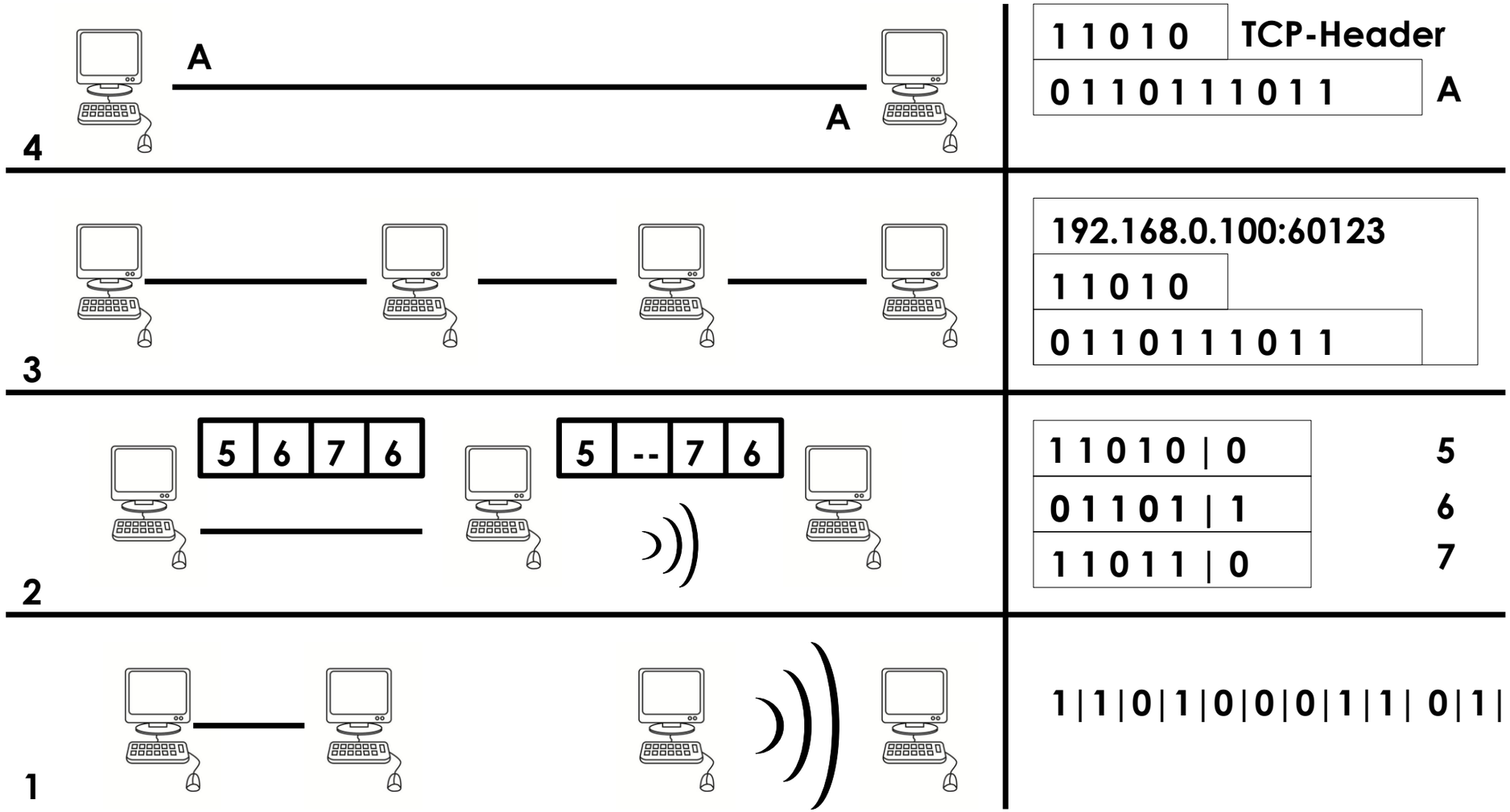
ÜBERSICHT

- Wiederholung
- TCP-Sockets
- UDP-Sockets
- Remote Method Invocation (RMI)

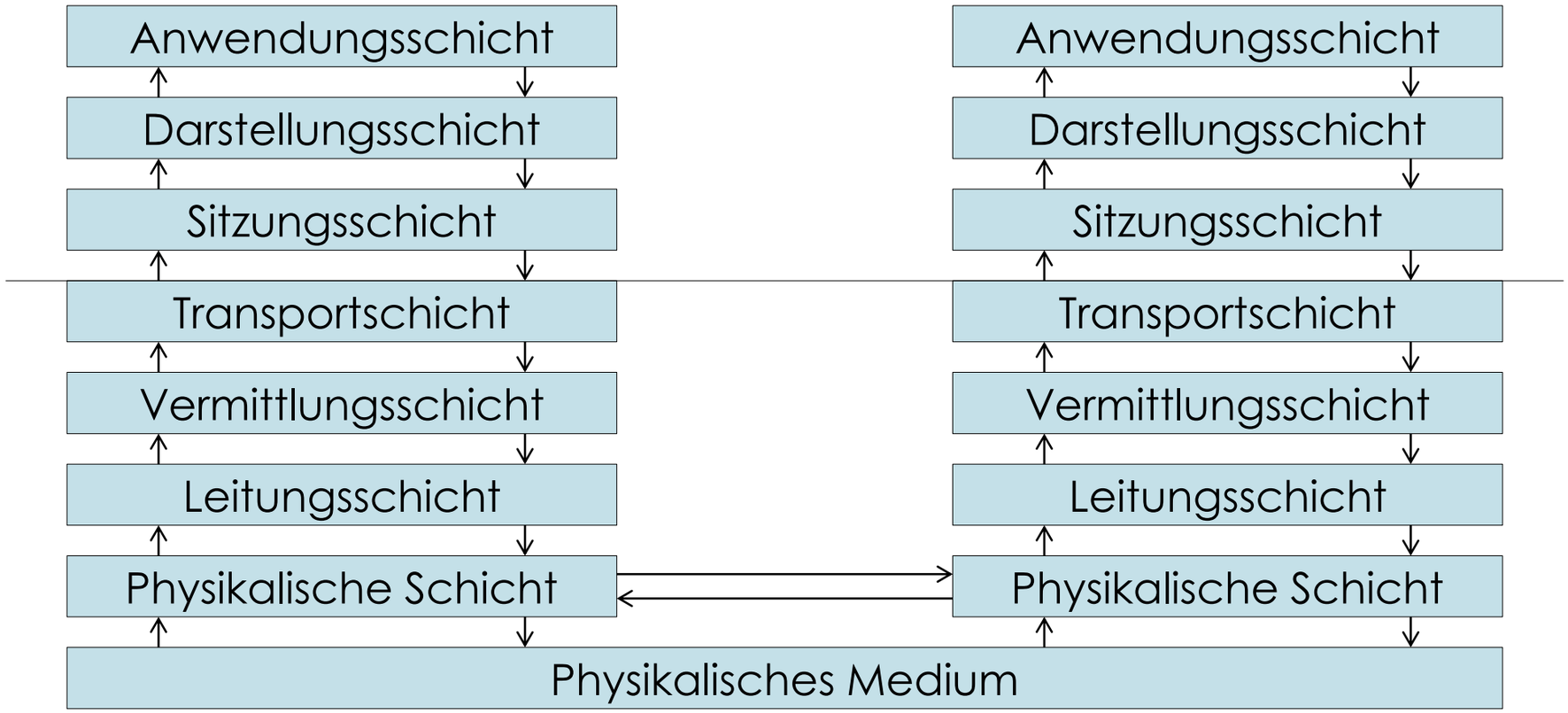
ISO/OSI-REFERENZMODELL



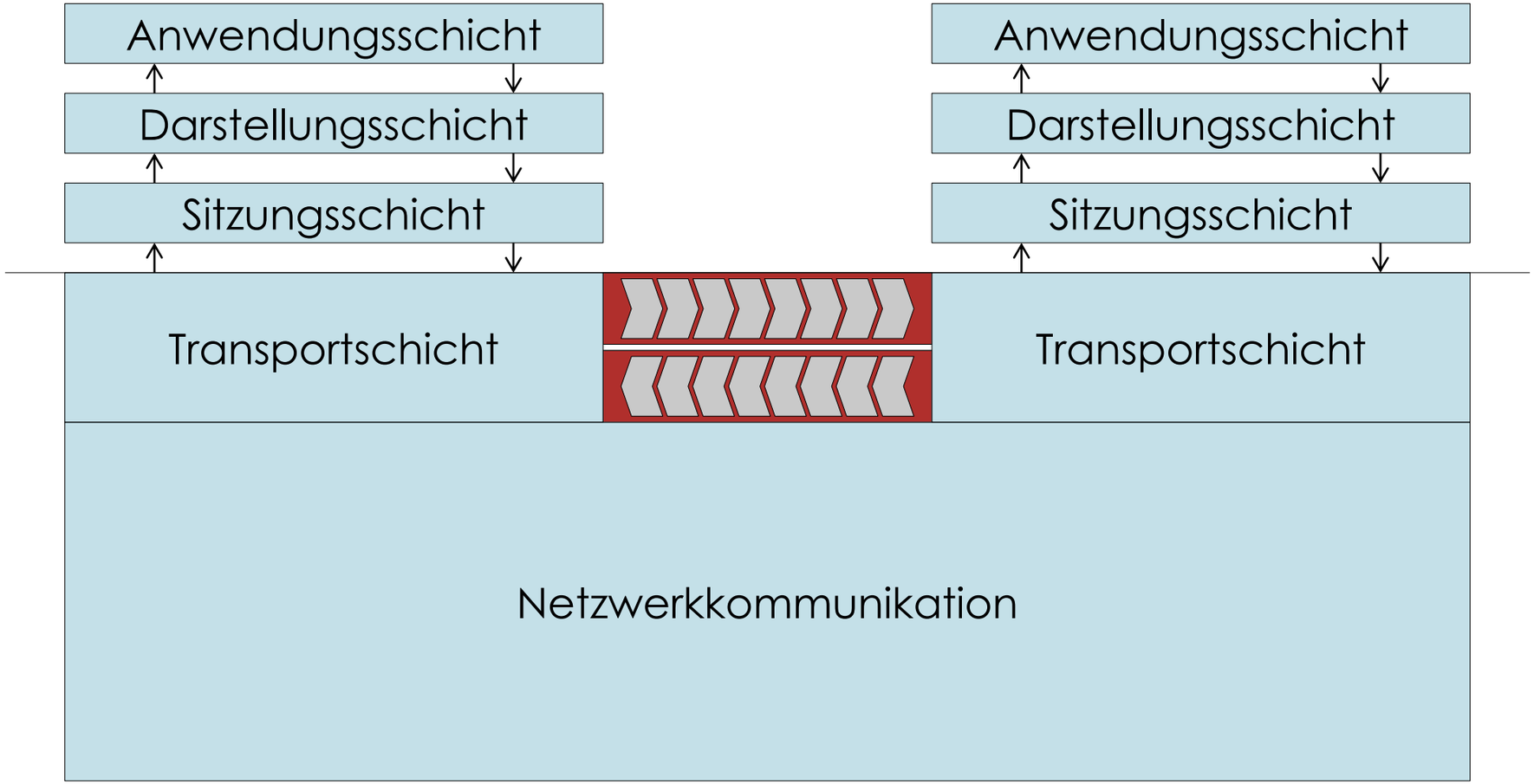
LOGISCHE KOMMUNIKATION



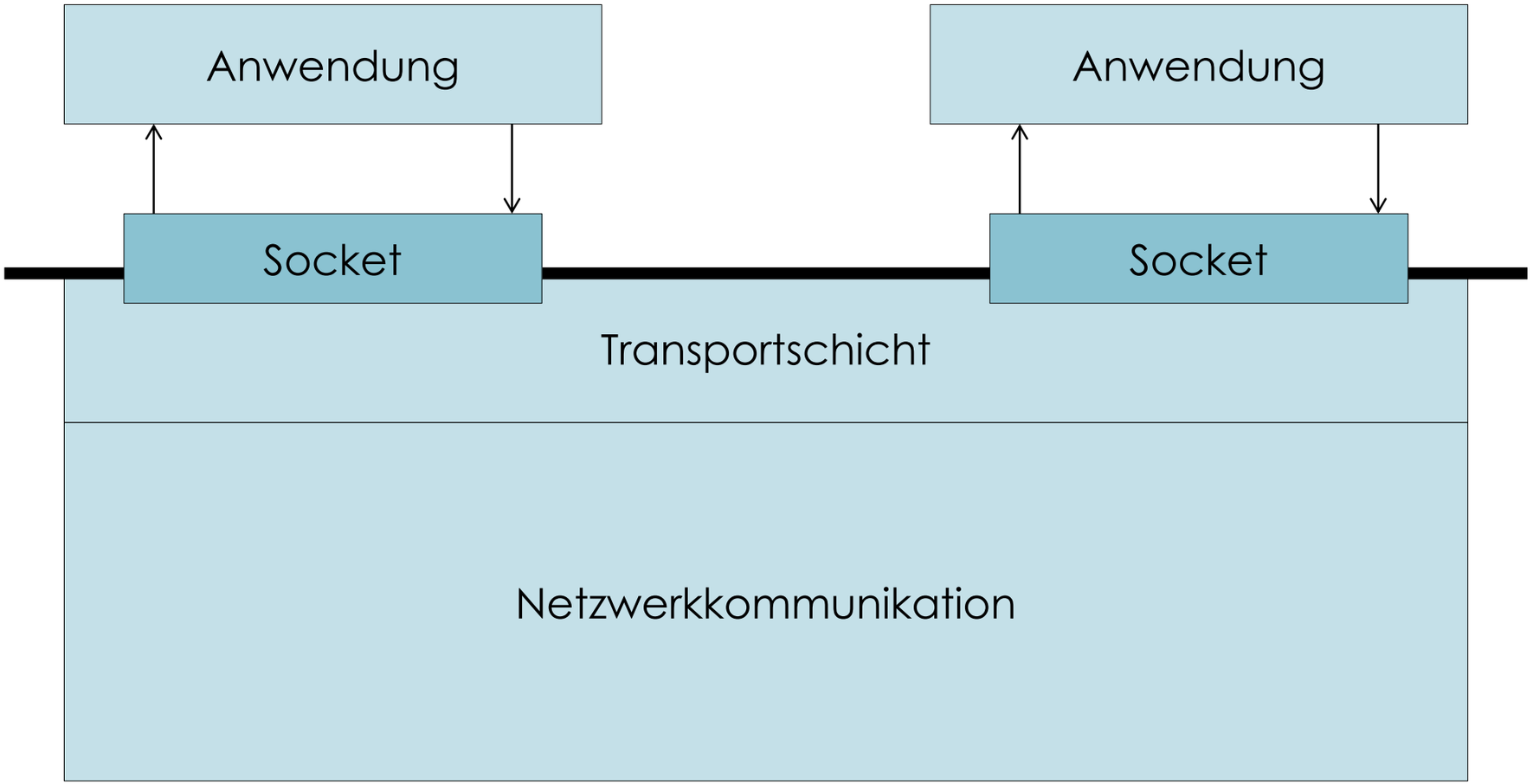
REFERENZMODELL



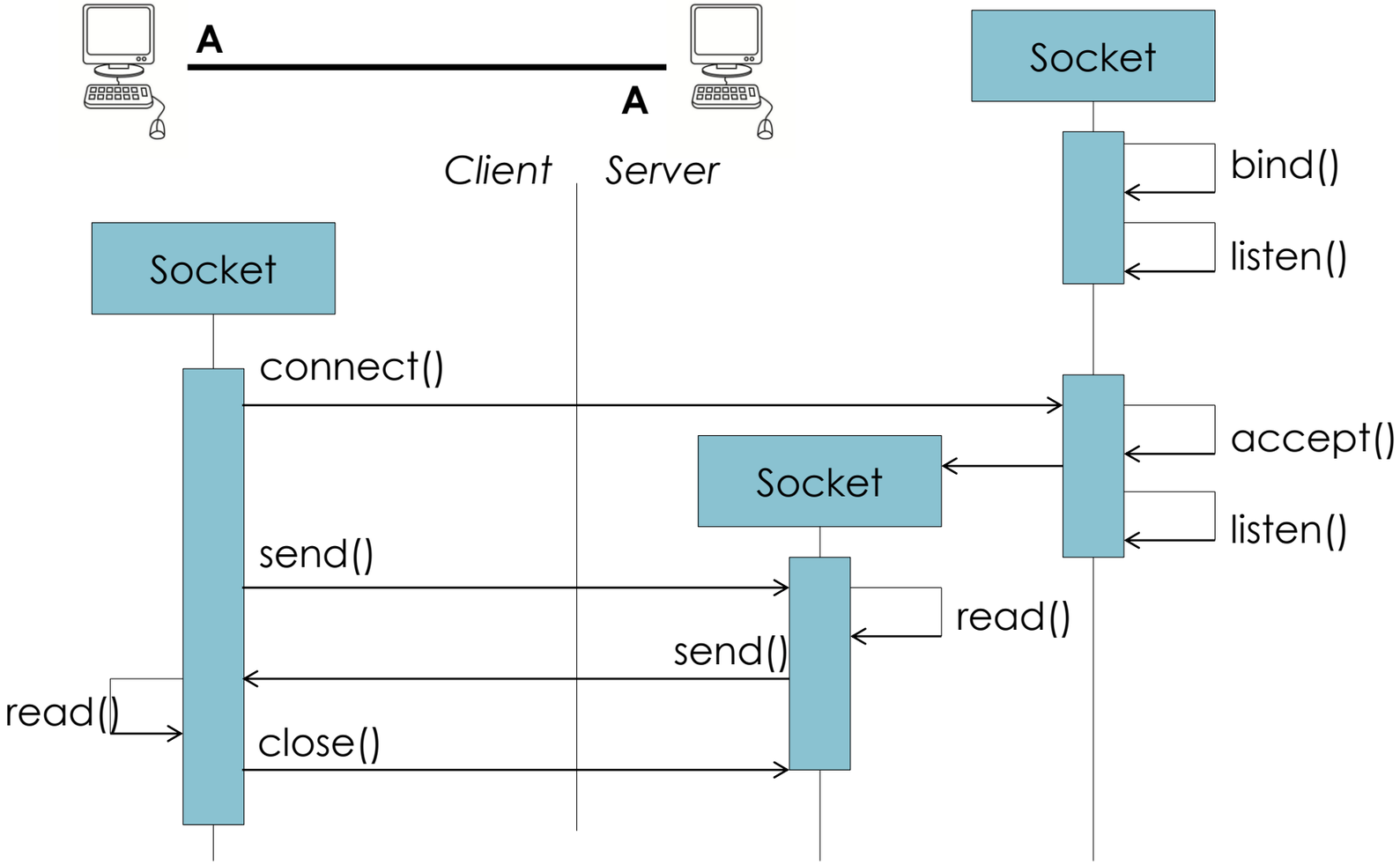
TRANSPORTSCHICHT



SOCKETS


Anwendung
Anwendung
Socket
Socket
Transportschicht
Netzwerkkommunikation

INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS



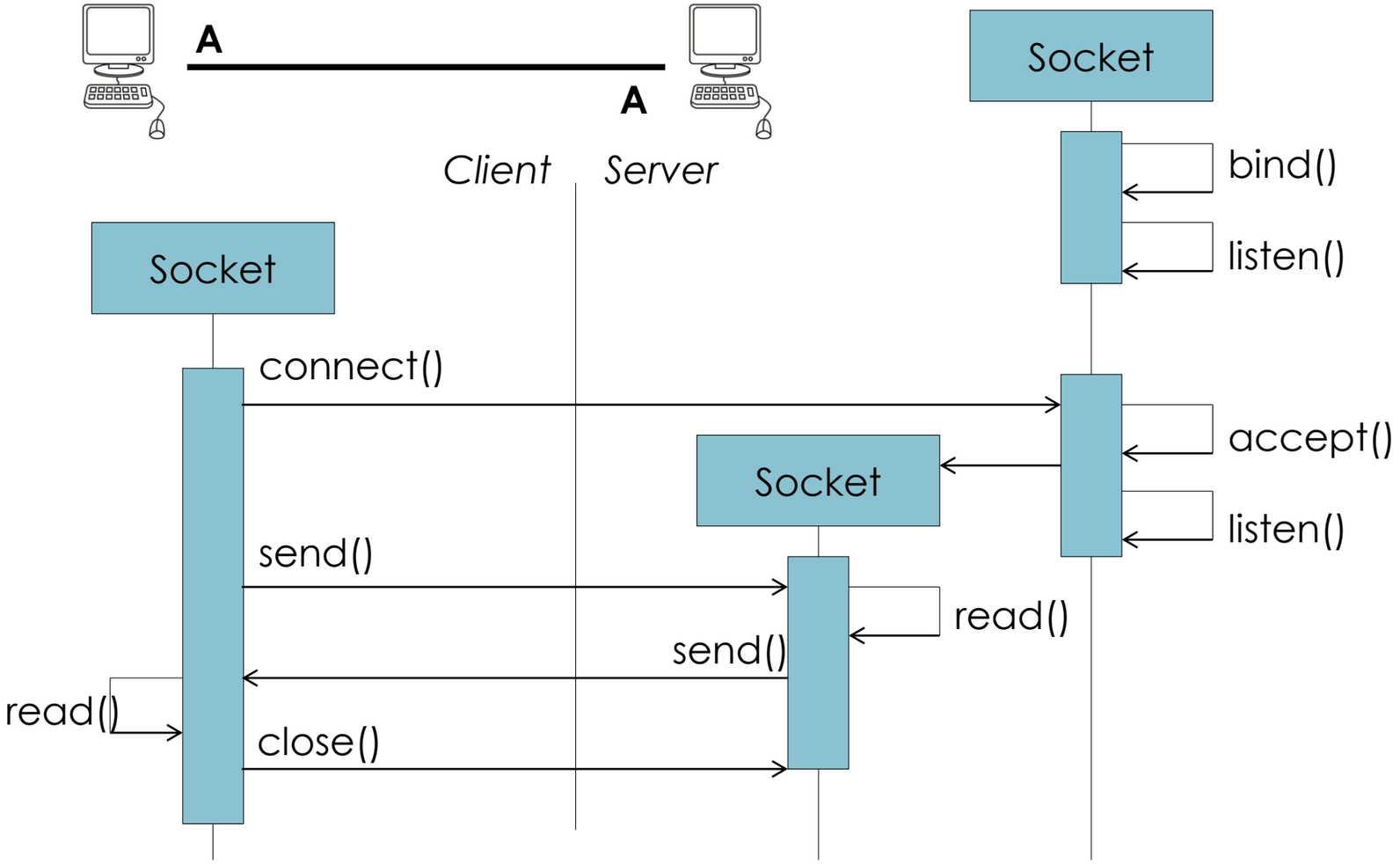
ÜBERSICHT

- Wiederholung
- TCP-Sockets
- UDP-Sockets
- Remote Method Invocation (RMI)

JAVA-SOCKETS

- Benötigte Klassen für Kommunikation im Paket: java.net
- Systemunabhängige Netzwerkkommunikation durch plattformunabhängig
- Implementierungen sowohl strombasiert (TCP),
  als auch paketorientiert (UDP) möglich

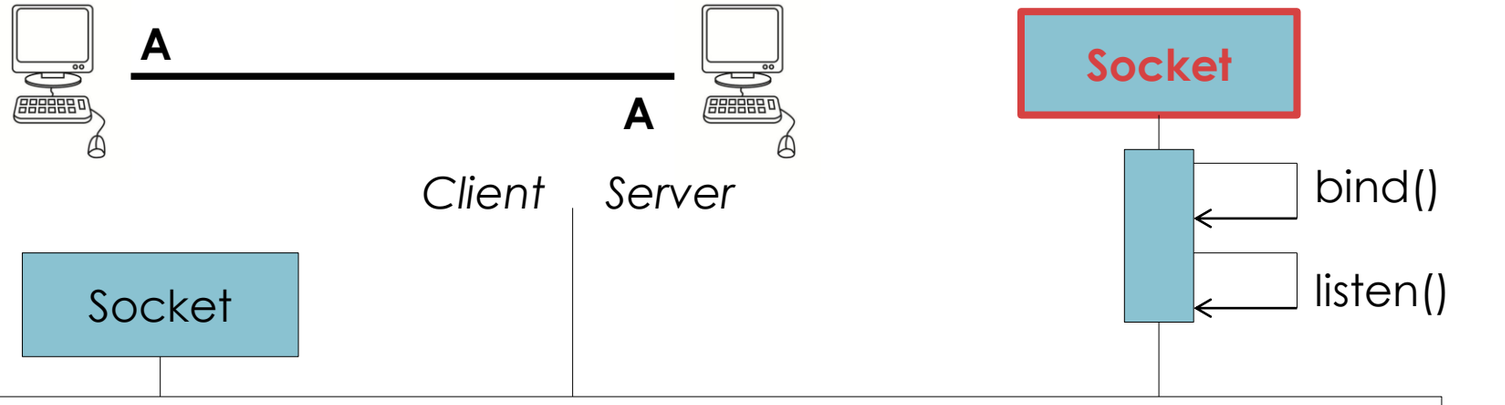
INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS



Snippet1

Backup

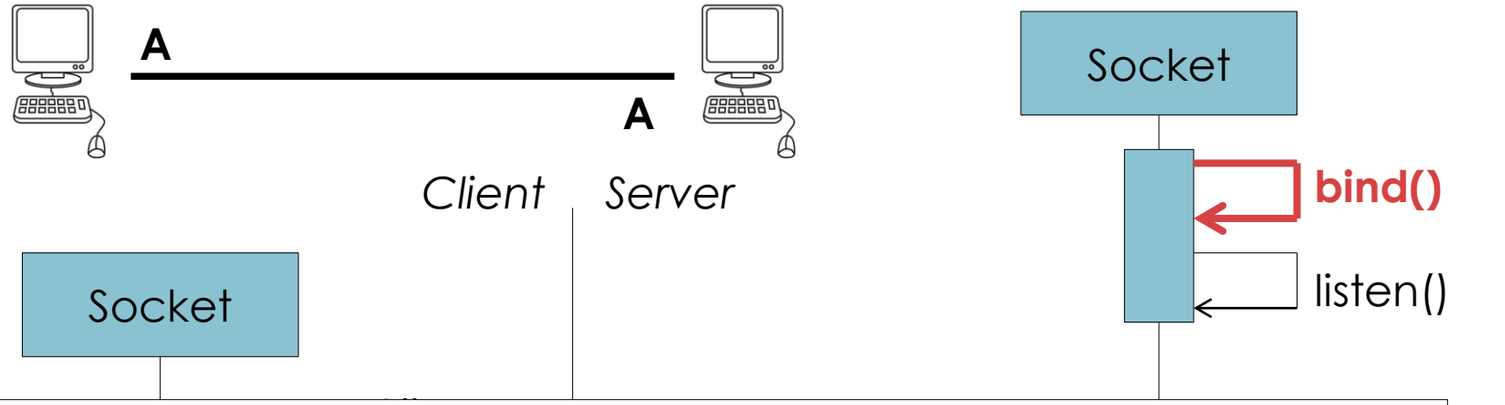
INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS




```
ServerSocket server = new ServerSocket();
```

Backup

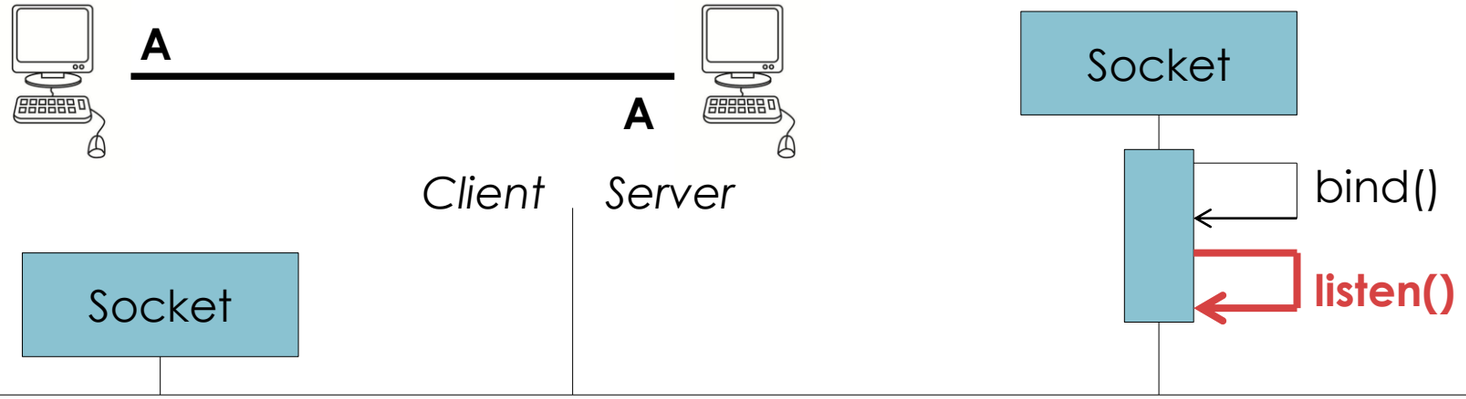
INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS




```
ServerSocket server = new ServerSocket();
server.bind(new InetSocketAddress());
```

Backup

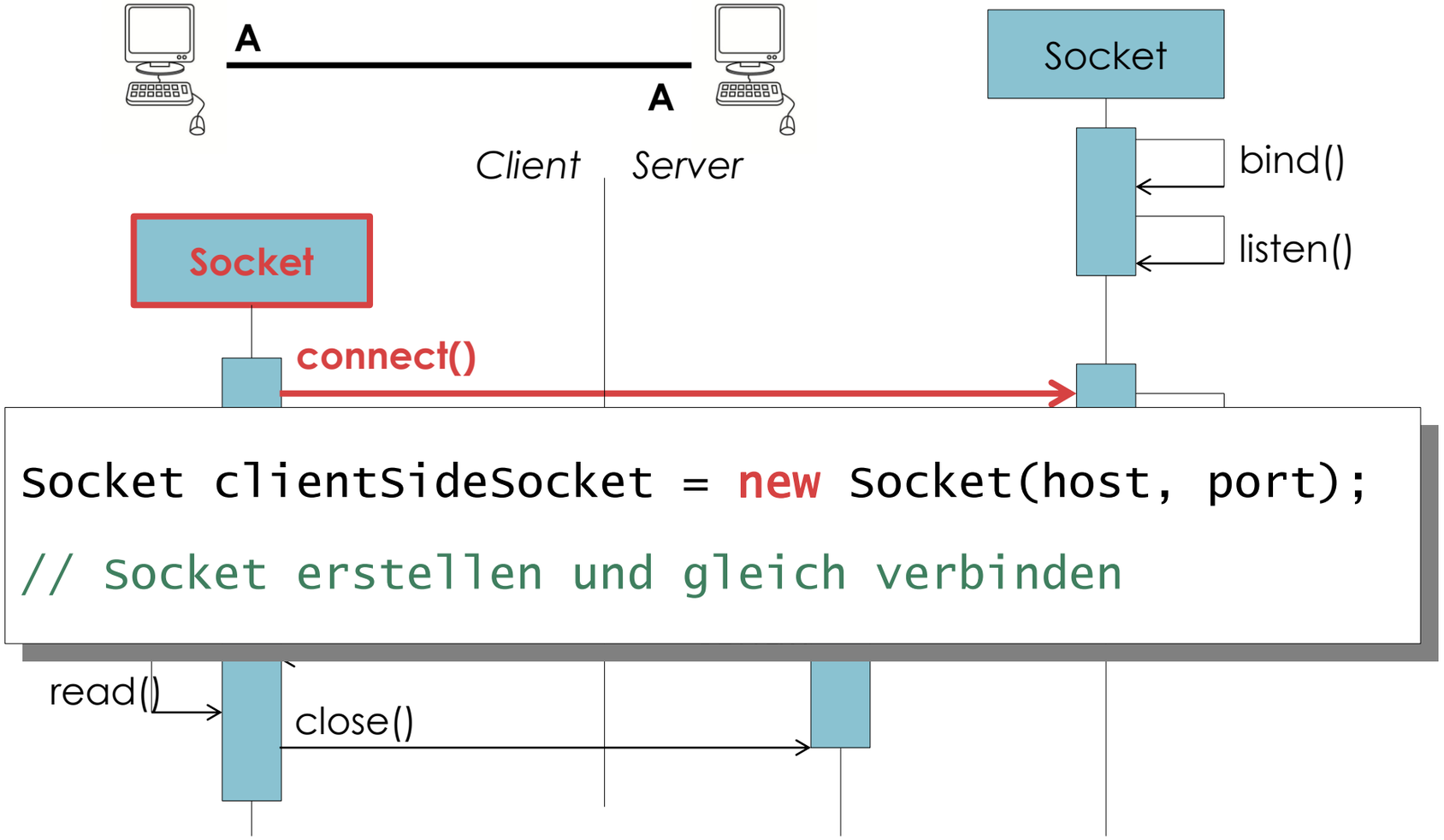
INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS




```
ServerSocket server = new ServerSocket();
server.bind(new InetSocketAddress());
Socket serverSideSocket = server.accept();
// Server wartet auf eingehende Verbindung
```

Backup

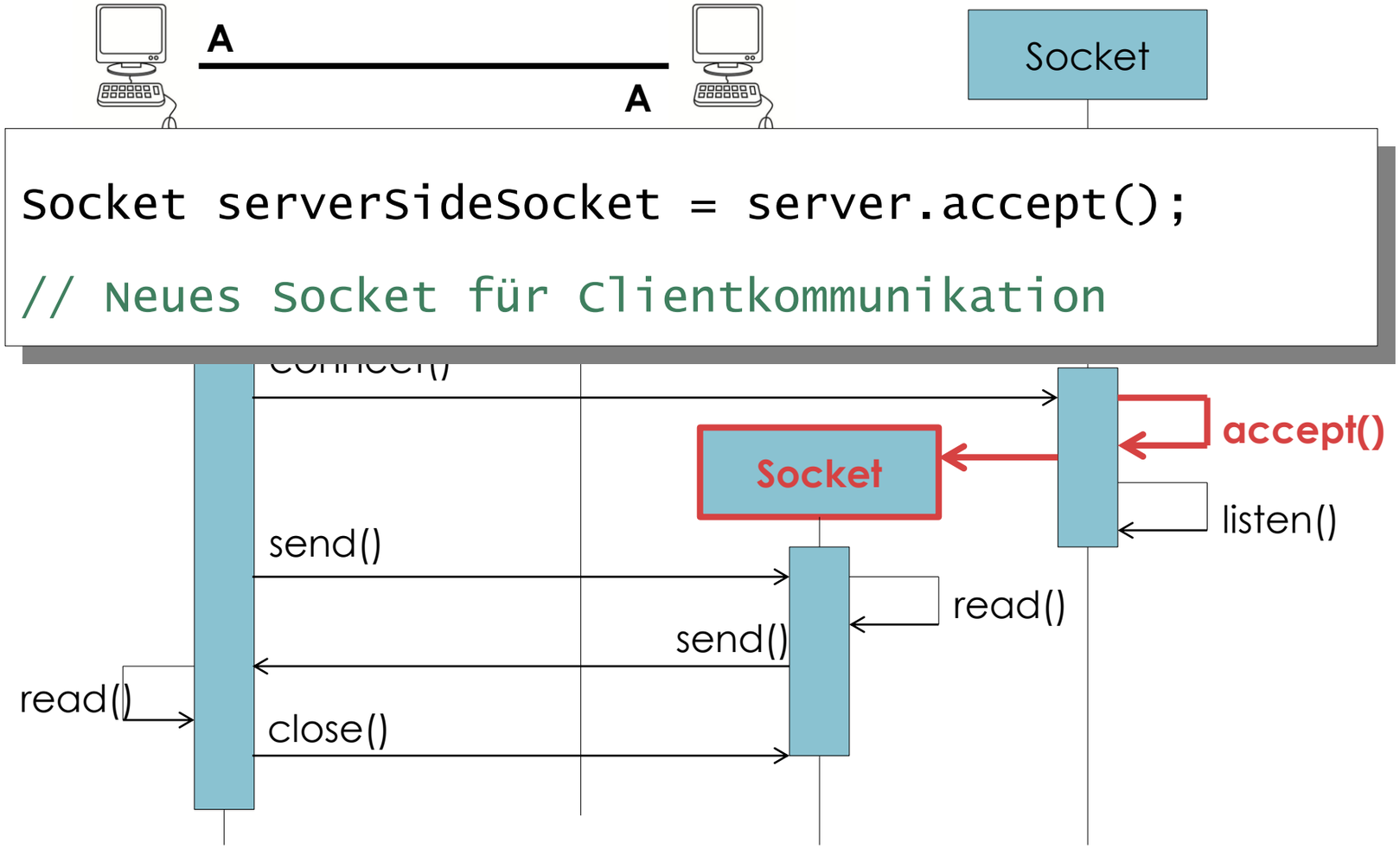
INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS




```
Socket clientSideSocket = new Socket(host, port);
// Socket erstellen und gleich verbinden
```

Backup

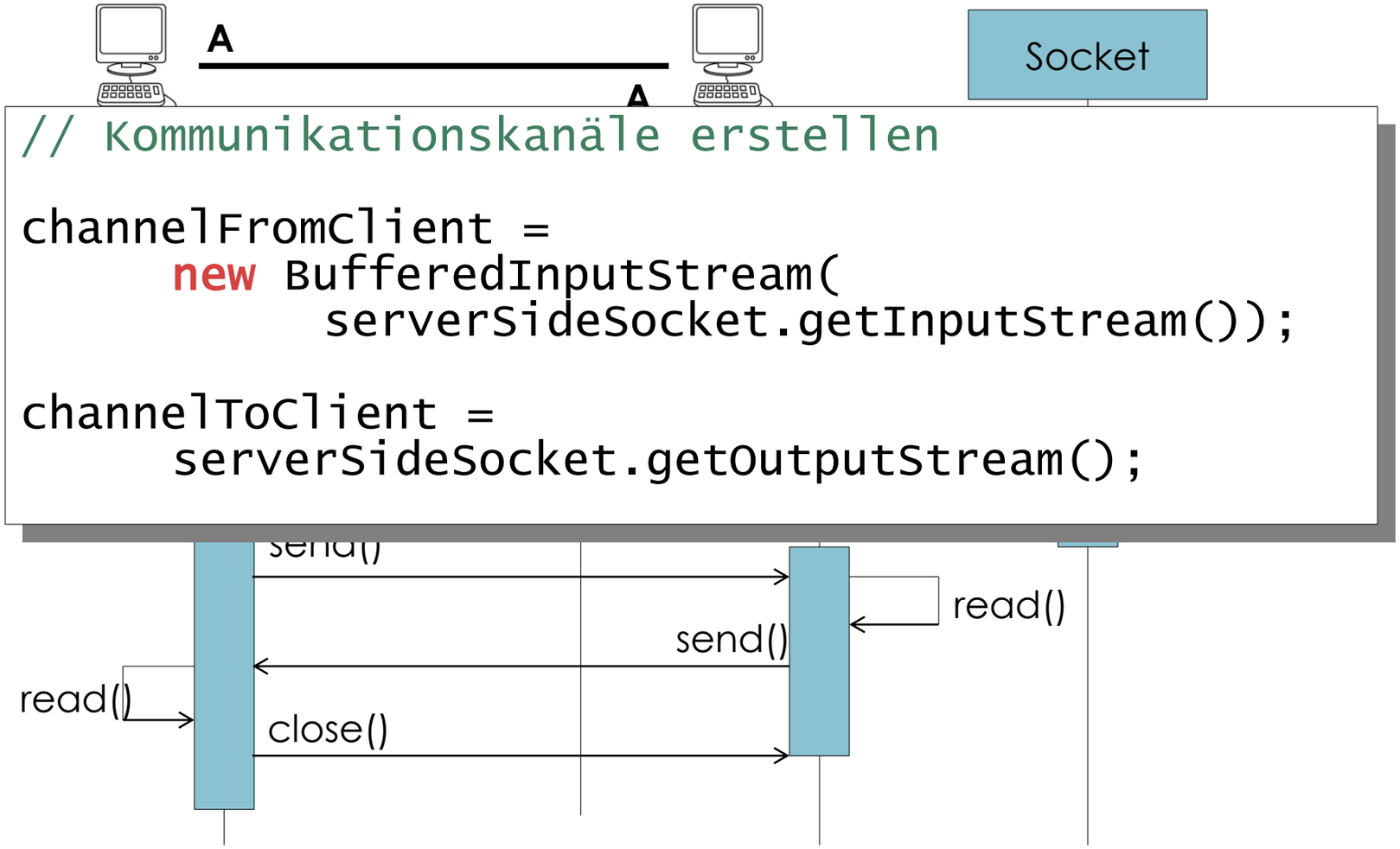
INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS




```
Socket serverSideSocket = server.accept();
// Neues Socket für Clientkommunikation
```

Backup

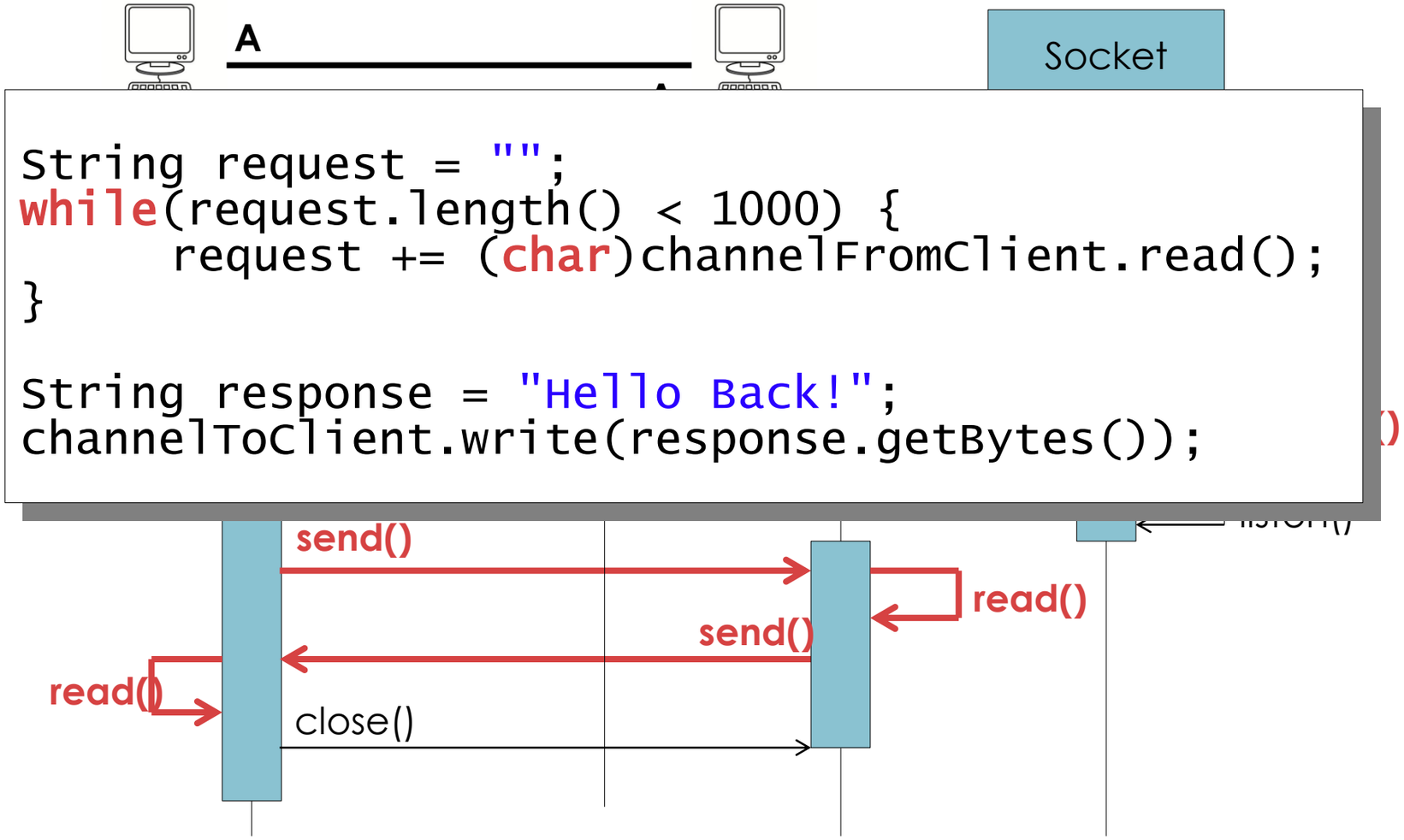
INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS




```
// Kommunikationskanäle erstellen

channelFromClient =
      new BufferedInputStream(
            serverSideSocket.getInputStream());

channelToClient =
      serverSideSocket.getOutputStream();
```

Backup

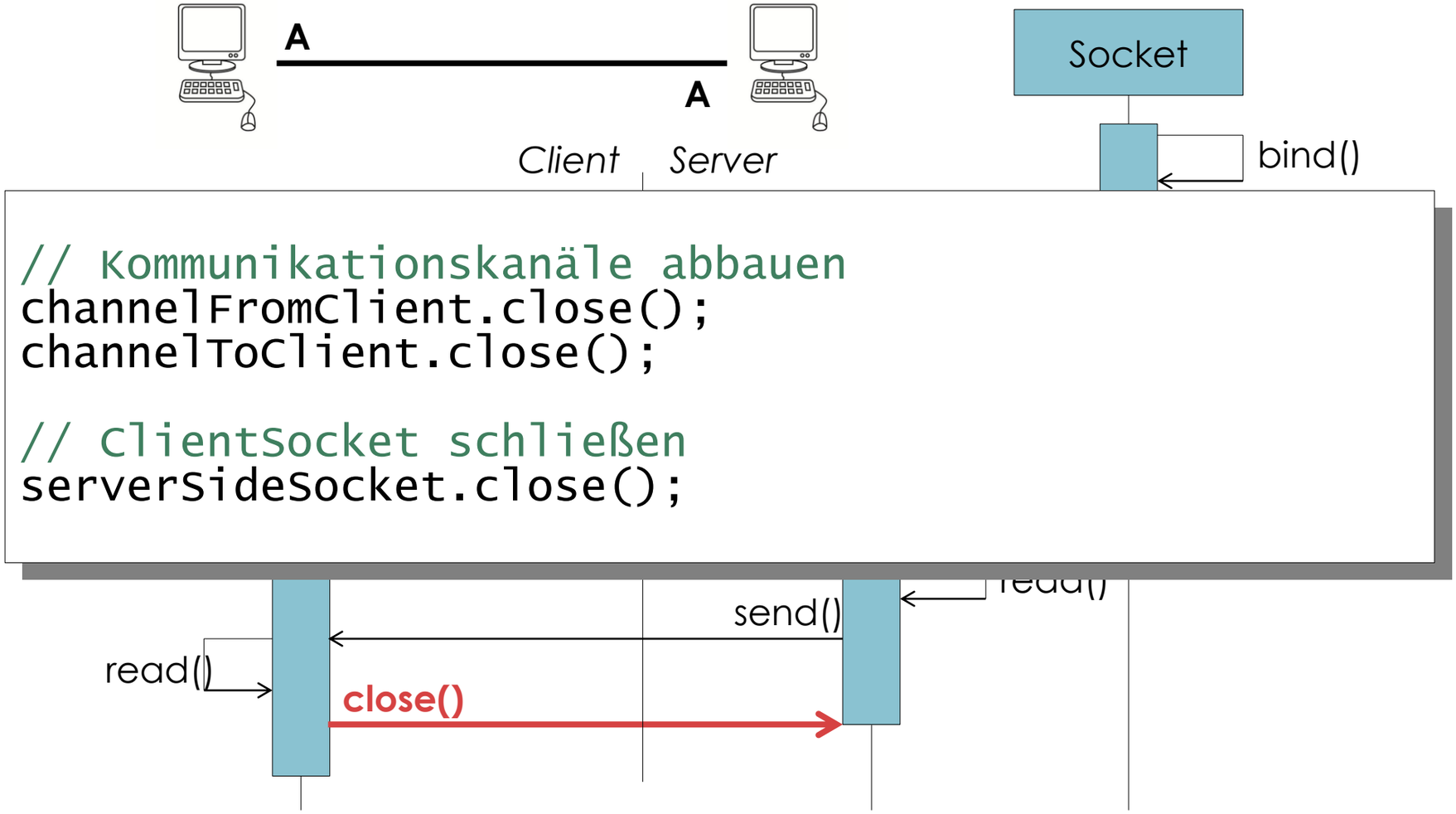
INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS




```
String request = "";
while(request.length() < 1000) {
      request += (char)channelFromClient.read();
}

String response = "Hello Back!";
channelToClient.write(response.getBytes());
```

Backup

INTERPROZESSKOMMUNIKATION MIT TCP SOCKETS

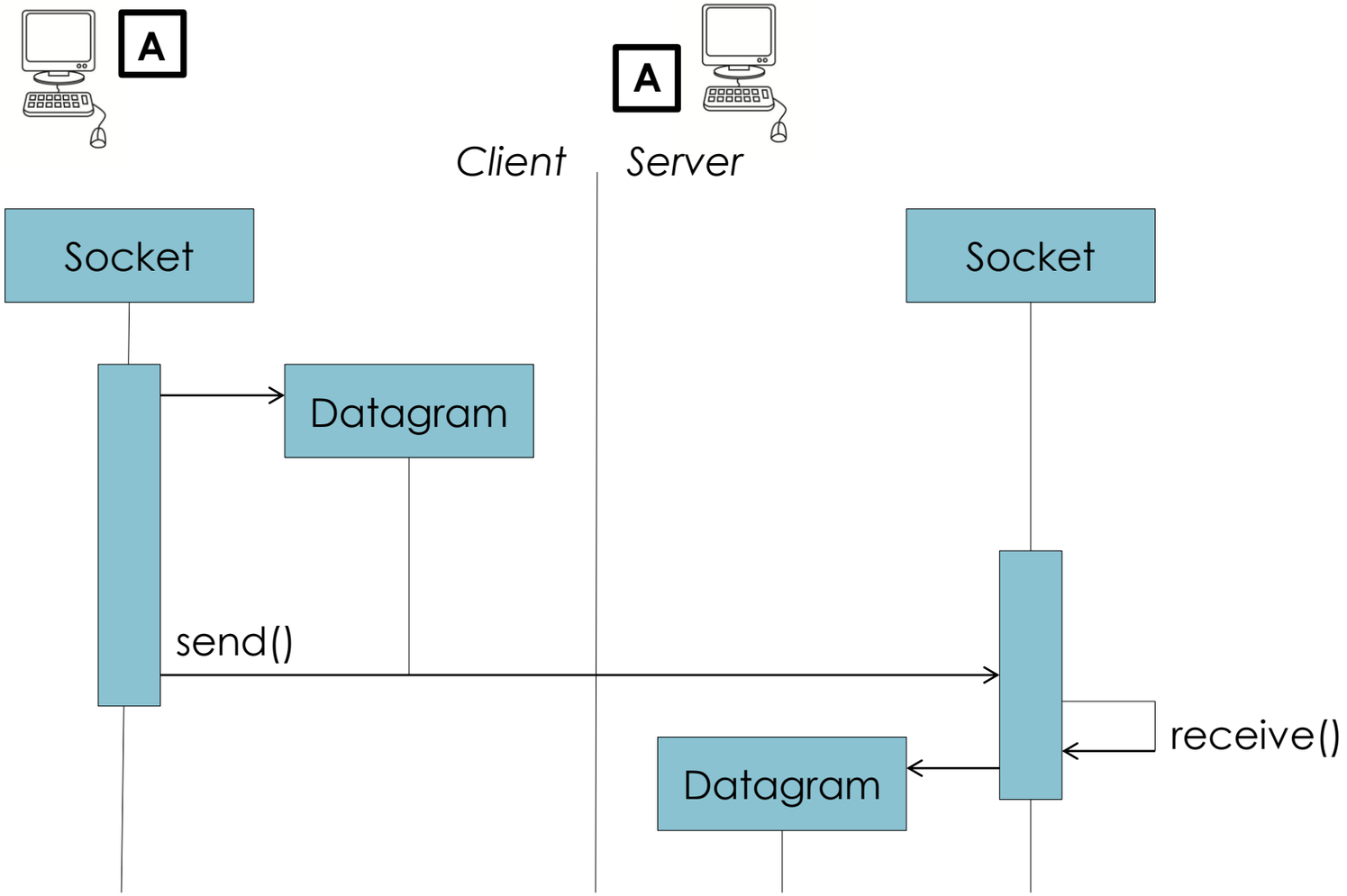



```
// Kommunikationskanäle abbauen
channelFromClient.close();
channelToClient.close();

// ClientSocket schließen
serverSideSocket.close();
```

ÜBERSICHT

- Wiederholung
- TCP-Sockets
- UDP-Sockets
- Remote Method Invocation (RMI)

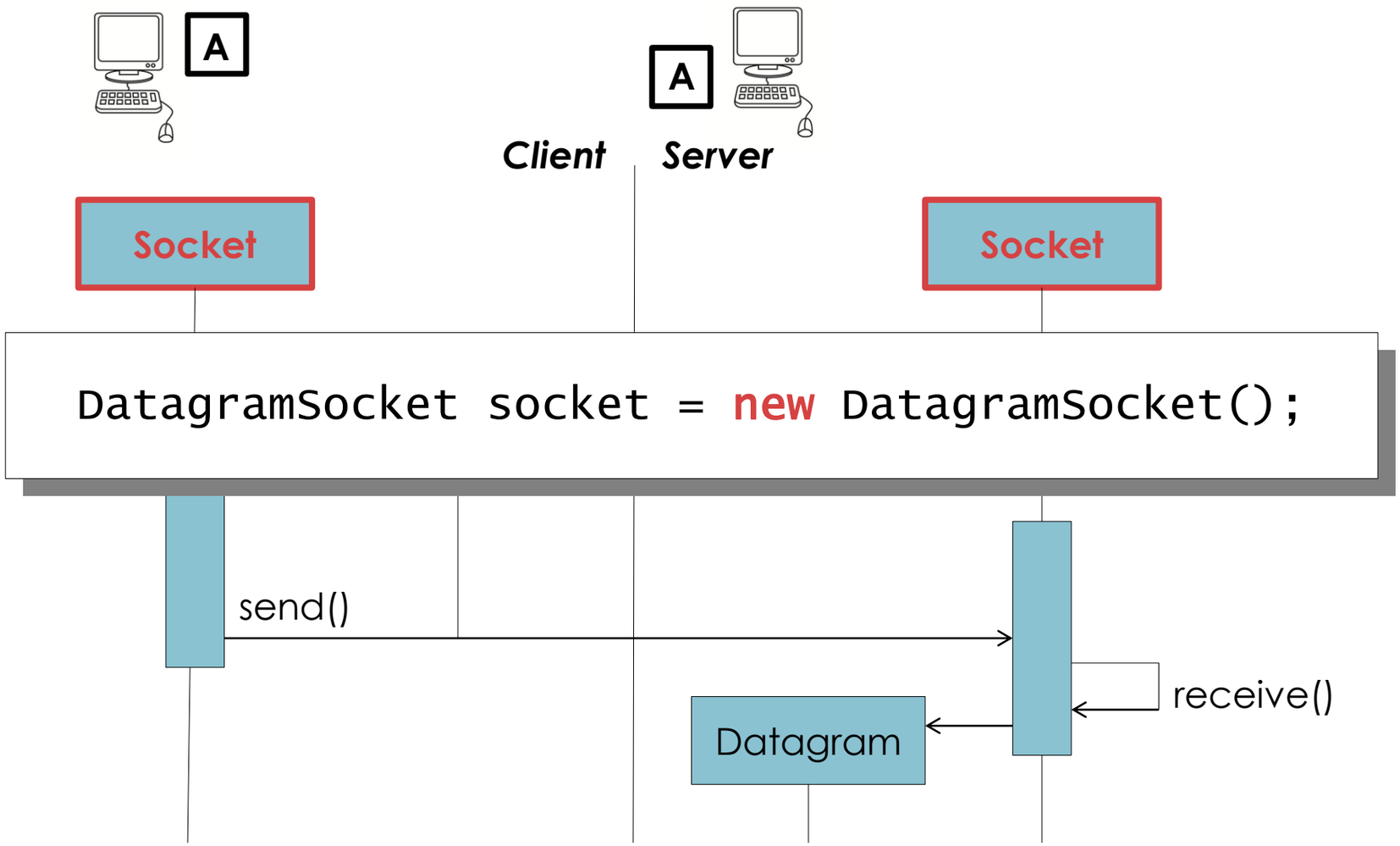
INTERPROZESSKOMMUNIKATION MIT UDP SOCKETS



Snippet2

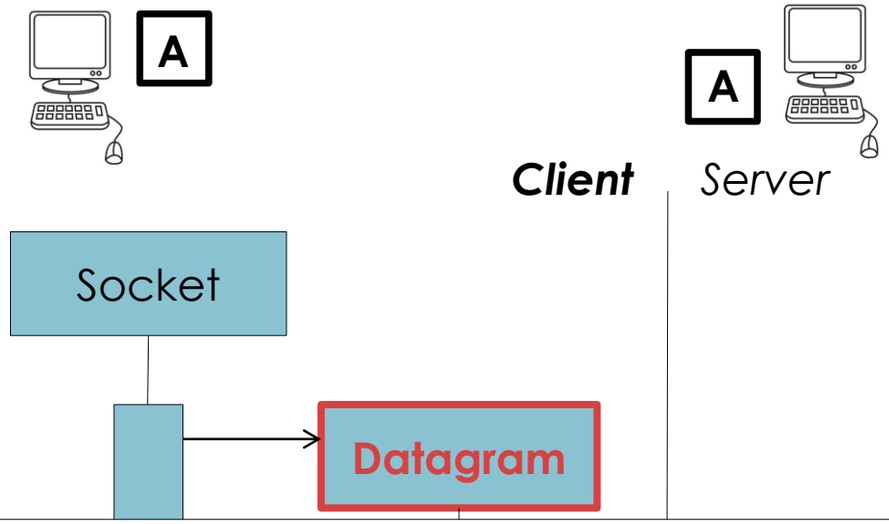
Backup

INTERPROZESSKOMMUNIKATION MIT UDP SOCKETS



Backup

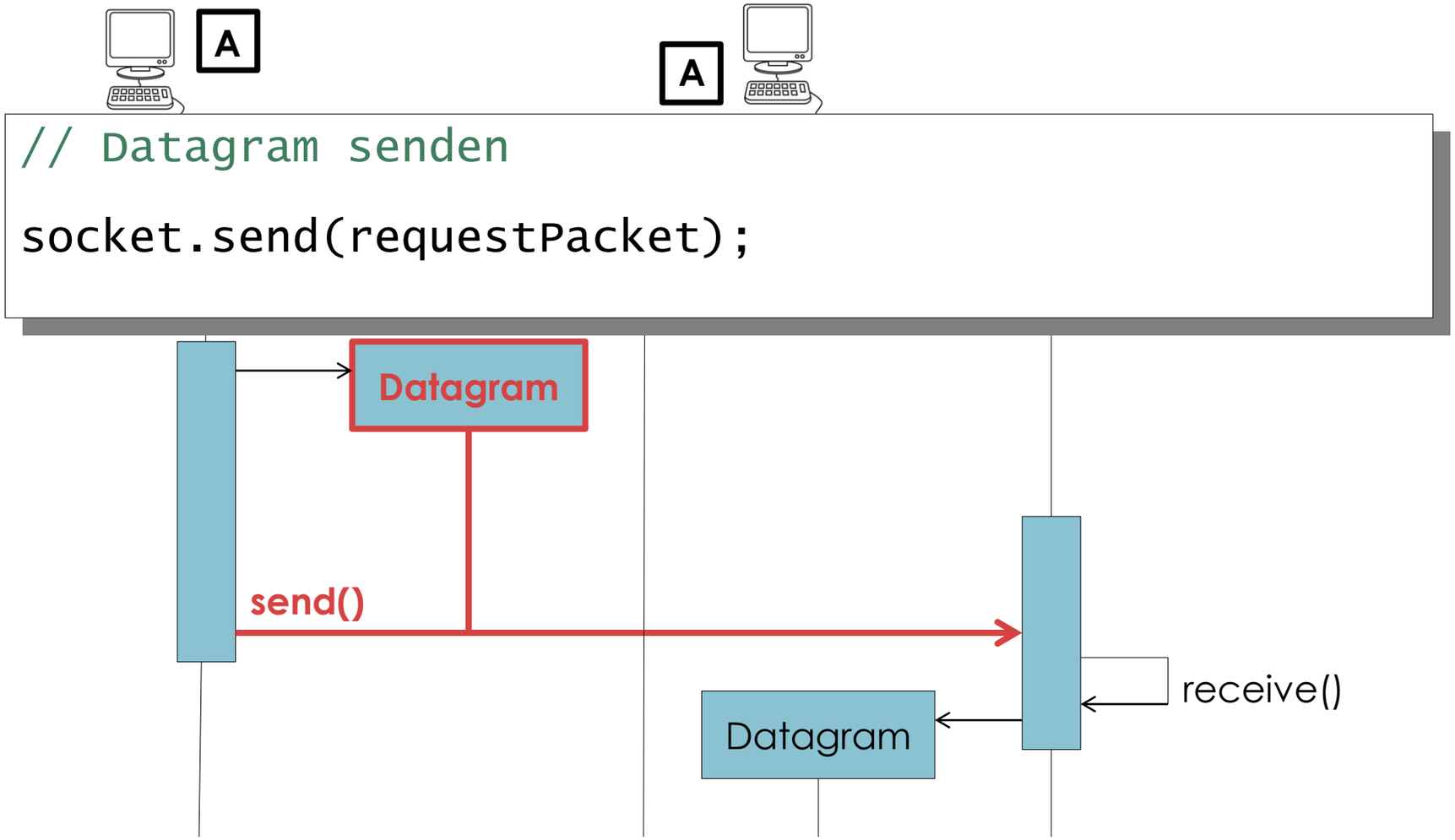
INTERPROZESSKOMMUNIKATION MIT UDP SOCKETS




```
String host = "localhost";
InetAddress address = InetAddress.getByName(host);

String request = "AAAand byeZZZ";
DatagramPacket requestPacket =
      new DatagramPacket(request.getBytes(),
            request.getBytes().length, address, port);
```

Backup

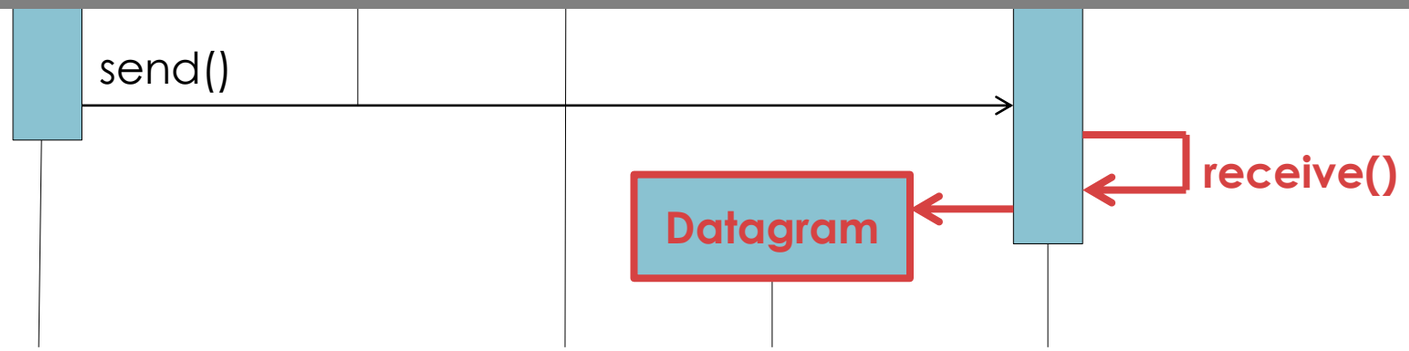
INTERPROZESSKOMMUNIKATION MIT UDP SOCKETS



Backup

INTERPROZESSKOMMUNIKATION MIT UDP SOCKETS

```
// Antwort Datagram erstellen und empfangen

DatagramPacket responsePacket =
        new DatagramPacket(new byte[1000], 1000);

socket.receive(responsePacket);
```



EINSCHRÄNKUNGEN SOCKET-KOMMUNIKATION

- Beschränkung auf Zeichenkettenströme
- Aufwändige Implementierung bei komplexen Protokollen
- Weitergabe von geworfenen Ausnahmen sehr schwierig
- Probleme bei der Übergabe von Werten komplexer Datentypen
- Nutzung der Objektserialisierung

BEISPIEL FÜR AUFRUF EINER FUNKTION ZUR ERMITTLUNG VON PRIMZAHLEN

- Für den Aufruf müssen übergeben werden
  - Ein Bezeichner der aufgerufenen Funktion
  - Die Intervallgrenze von, als ganze Zahl
  - Die Intervallgrenze bis, als ganze Zahl
- Für die Rückgabe müssen übertragen werden
  - Eine beliebig lange Liste von einzelnen ganzen Zahlen
  - Eine mögliche leere Liste
- Die Funktion kann zudem sehr lange oder sehr kurz laufen
- Sinnvolle asynchrone Kommunikation
- Unterteilung einer großen Rückgabe-Liste in mehrere Blöcke

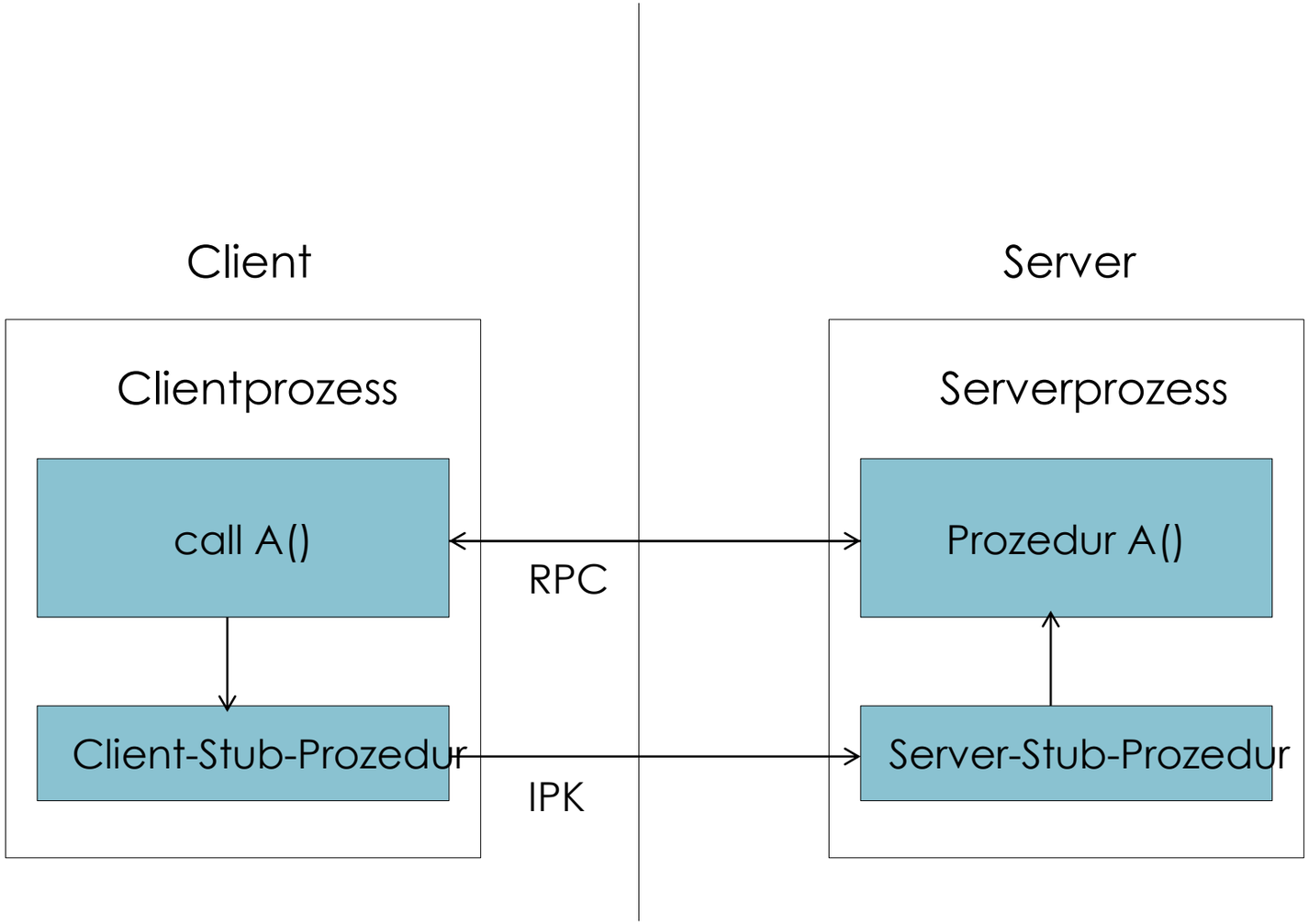
ÜBERSICHT

- Wiederholung
- TCP-Sockets
- UDP-Sockets
- Remote Method Invocation (RMI)

AUFRUF VON VERTEILTEN PROZEDUREN (RPC)

- Sockets sind die niedrigste Form der Interprozesskommunikation (IPK)
- Interpretation der Nachrichten muss von Hand implementiert werden
- Entwicklung komplexerer Funktionen ist aufwändig und fehleranfällig
- IPK besser durch expliziten Aufruf von entfernten Prozeduren
- Konzept des Remote Procedure Call (RPC)
- Beruht auf lokalen Stellvertretern (Stubs)
- Simulieren normalen synchronen Prozeduraufruf
  - Leiten den Aufruf über Sockets an Server weiter
  - Server nimmt dies ebenfalls als normalen Prozeduraufruf war
  - Somit zuverlässige Syntax- und Aufrufbehandlung
  - Ggf. Typkonvertierungen und Formatanpassungen

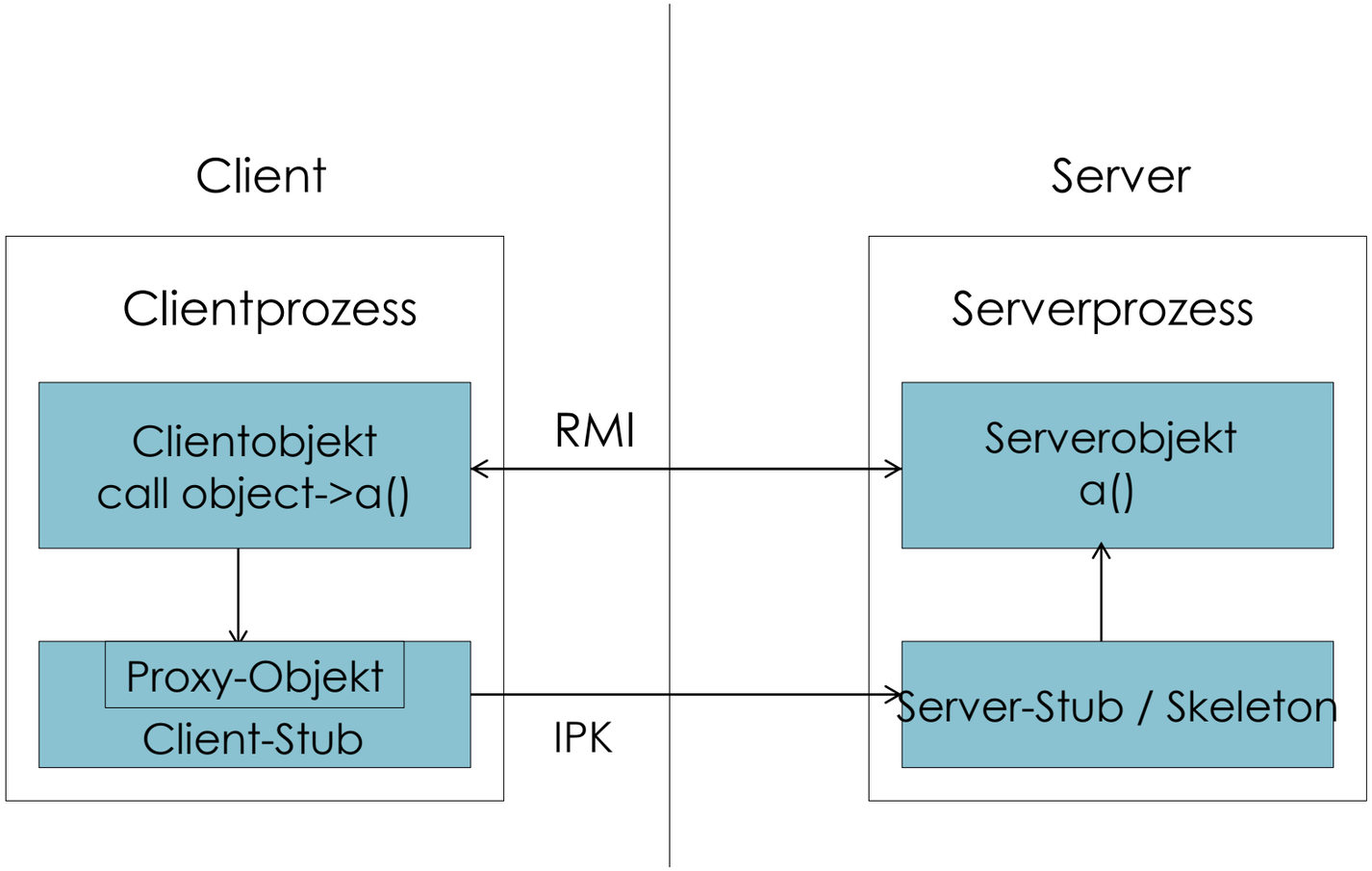
RPC - ABLAUF


Client
Server
Clientprozess
Serverprozess
call A()
Prozedur A()
RPC
Client-Stub-Prozedur
Server-Stub-Prozedur
IPK

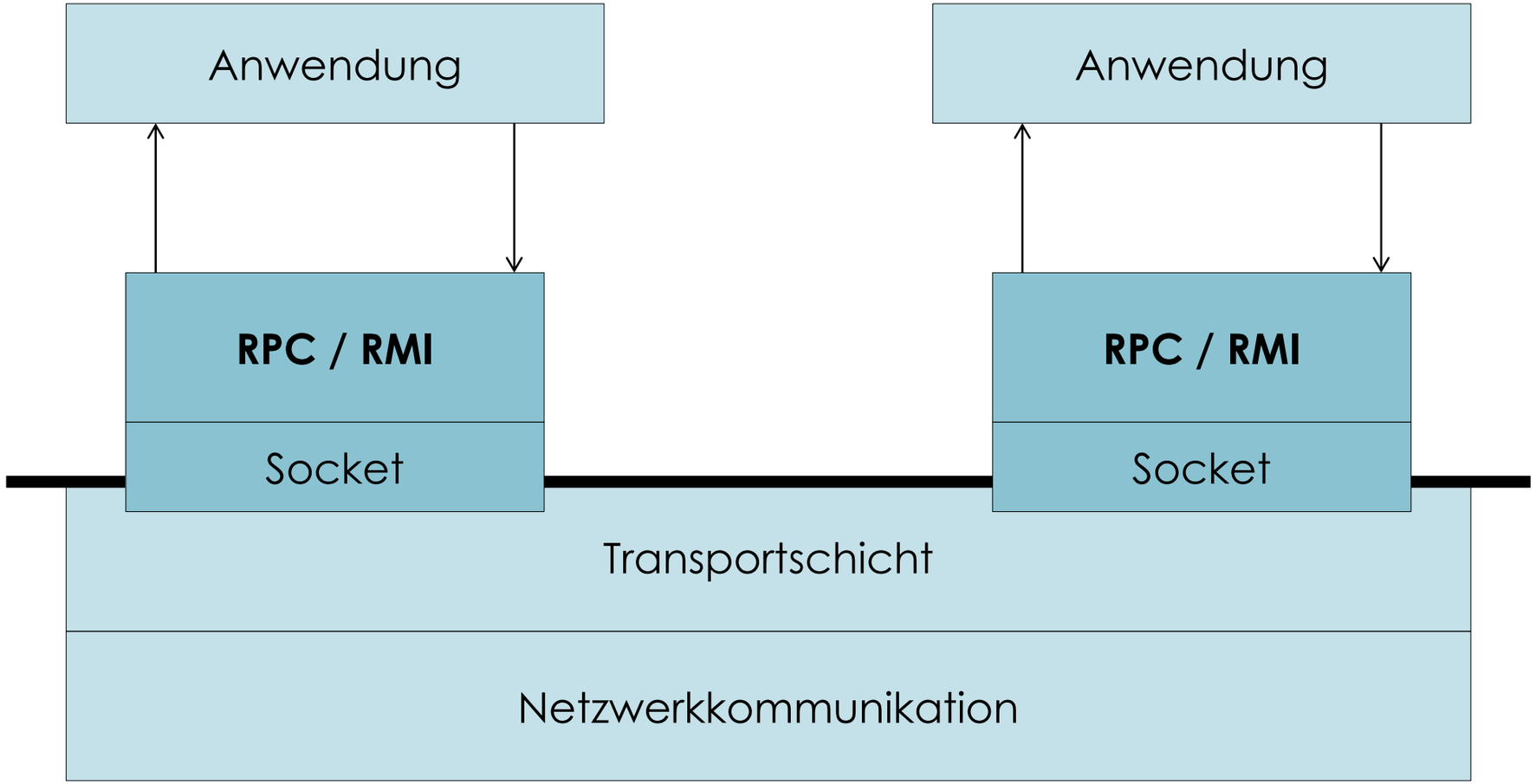
REMOTE METHOD INVOCATION (RMI)

- Objektorientierte Variante des *Remote Procedure Call* (RPC) in Java
- Konzept auf verteilten Objekten mit aufrufbaren Methoden
- Synchrone Kommunikation mit Aufruf und Ergebnisrückgabe
- Nutzt ebenfalls Stubs für transparente Kommunikation
- Zusätzlich auf Clientseite ein Stub-Objekt, genannt Proxy
- Simuliert das entfernte Objekt
- Server-Stub wird auch als Skeleton bezeichnet

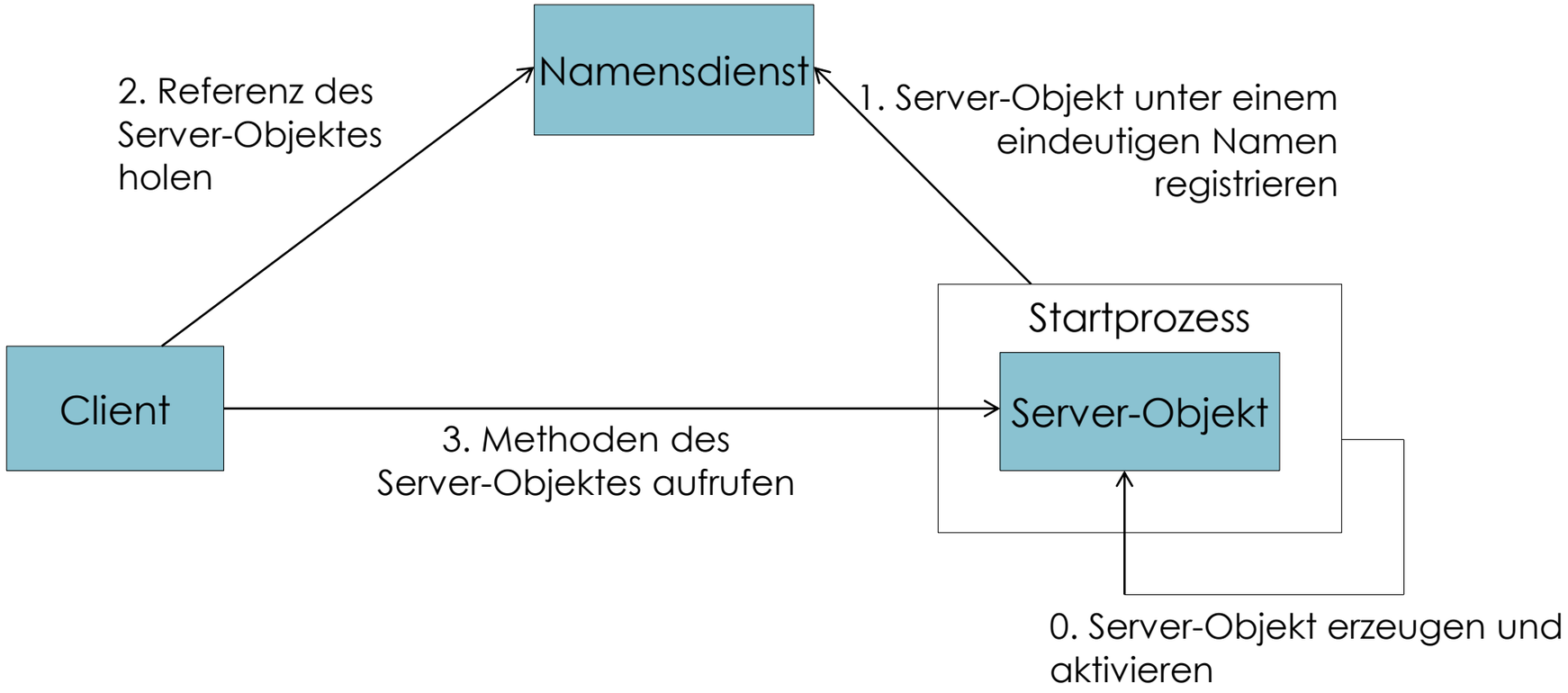
REMOTE METHOD INVOCATION (RMI)



RPC/RMI UND SOCKETS



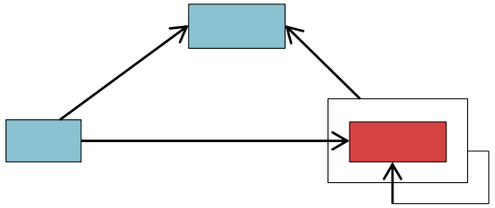
AUFBAU RMI



Snippet3

Backup

RMI – SERVERSEITE - REMOTE-INTERFACE

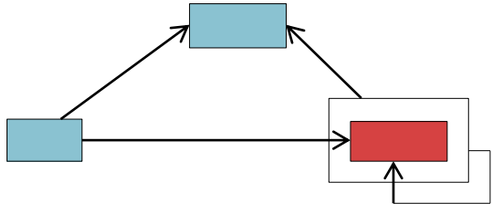


- Definition des Server-Dienstes als Schnittstelle
- Schnittstelle muss von `java.rmi.Remote` erben
- Somit ein sogenanntes *Remote-Interface*
- Methodensignaturen müssen `java.rmi.RemoteException` werfen

```
public interface MyRemoteInterface extends Remote{
        int myRemoteMethod() throws RemoteException;
}
```

Backup

RMI – SERVERSEITE – REMOTE-IMPLEMENTIERUNG

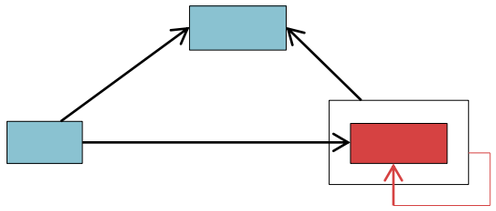


- Verteilte Objekt implementiert Remote-Interface
- Kann zusätzlich auch lokale Implementierung enthalten

```
public class MyRemoteClass implements MyRemoteInterface

        // Methodenimplementierung
        int myRemoteMethode() throws RemoteException  {

        }

        ...
}
```

Backup

RMI – SERVERSEITE – VERTEILTES OBJEKT ERZEUGEN

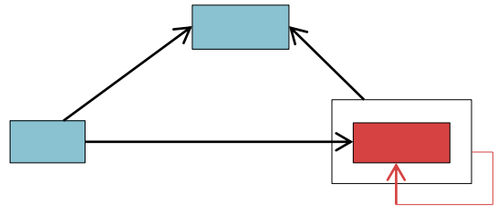


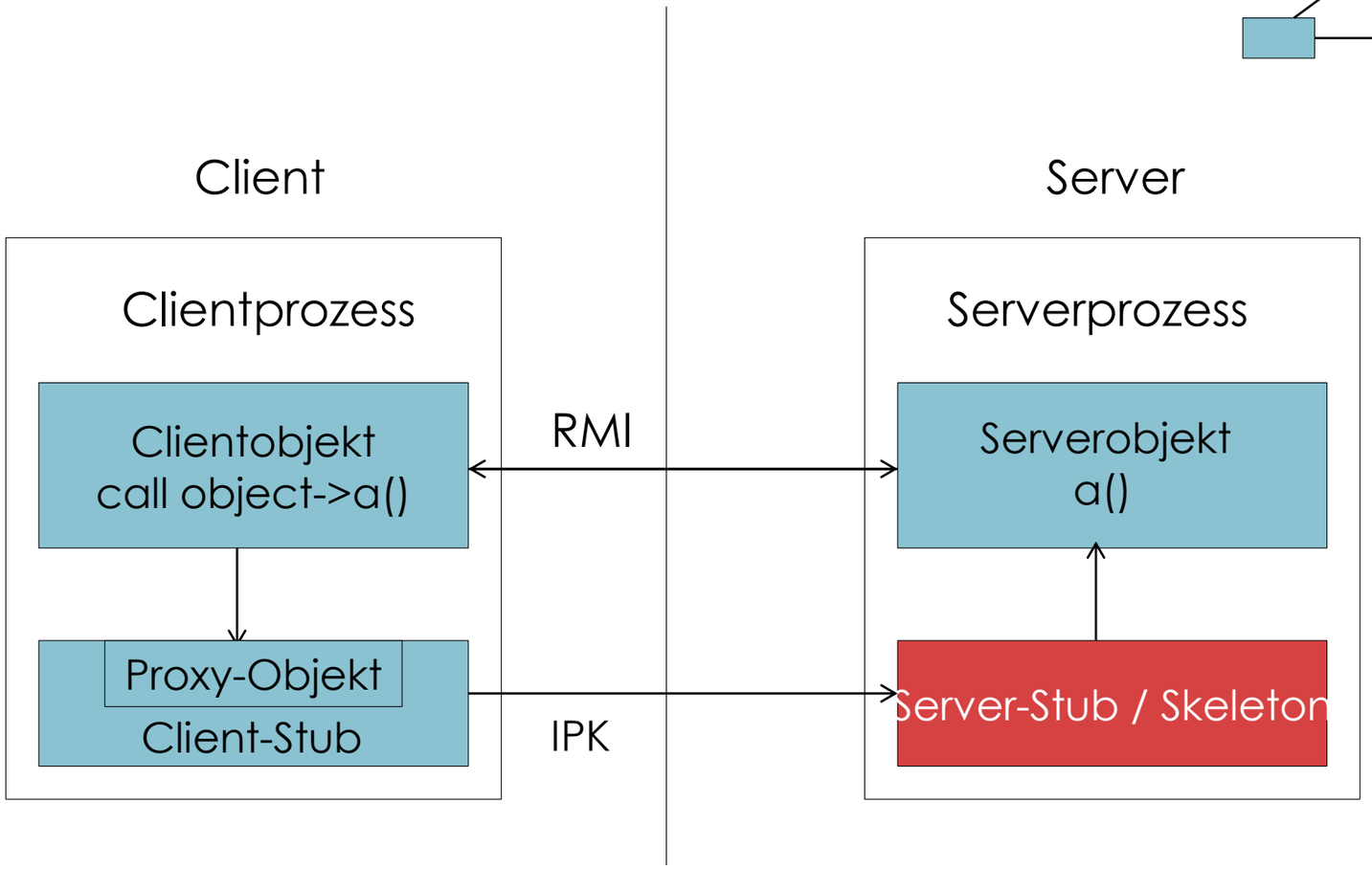
- Erzeugen des verteilten Objekts mit statischen Methode `UnicastRemoteObject.exportObject()`
- Erzeugt ein dynamisches Proxy-Objekt
- Per TypeCast Verwendung als Objekt

```
...
//Instanziierung
MyRemoteClass myRemoteObject = new MyRemoteClass();

...

//Erzeugung eines Proxy
MyRemoteInterface remoteInterface = (MyRemoteInterface)
      UnicastRemoteObject.exportObject(myRemoteObject,0);
...
```

Backup

REMOTE METHOD INVOCATION (RMI)

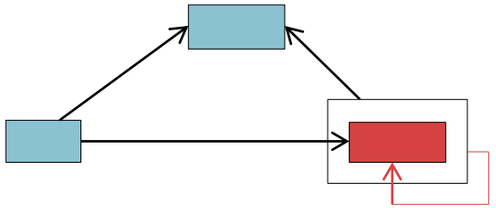




Backup

RMI – SERVERSEITE – VERTEILTES OBJEKT ERZEUGEN

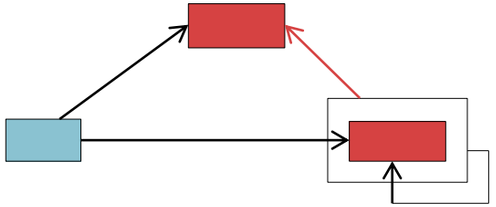
- Alternativ kann Serverobjekt auch von *UnicastRemoteObject* erben
- Automatischer Export neu erzeugter Objekte



```
public class MyRemoteClass
      extends UnicastRemoteObject
      implements MyRemoteInterface {

      ...
      // Methodenimplementierung
      int myRemoteMethode() throws RemoteException  {

      }
      ...
}
```

Backup

RMI – SERVERSEITE – OBJEKT REGISTRIEREN

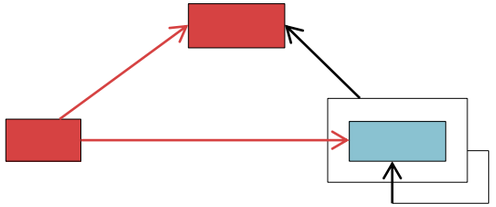


- Anbindung an die RMI - Registry
- Erzeugen einer neuen lokalen Registry
- Binden des verteilten Objekts an eine RMI-Registry

```
...
//Registry erzeugen auf Localhost, Port 1099
LocateRegistry.createRegistry(Registry.REGISTRY_PORT);

...
//Binden des dynamischen Proxy Objekts
Naming.rebind(„myRemoteService“, myRemoteObject);

...
//Binden des von UnicastRemoteObject erbenden Objekts
Naming.rebind(„myRemoteService“, new MyRemoteClass());
```

Backup

RMI - CLIENTSEITE



- Beschaffen einer Referenz zum entfernten Objekt
- Anfrage an einer Registry zum Beispiel mit
  `Naming.lookup("//<hostname>/<Servicebezeichnung>")`
- Casten der Objektreferenz zu RemoteInterface
- Entfernter Methodenaufruf auf Serverobjekt wie bei lokalem Objekt

```
...
// Ermittlung der Referenz auf das verteilte Objekt
MyRemoteInterface myRemoteObject = (MyRemoteInterface)
       Naming.lookup("//host/myRemoteService");
...

// Normaler Methodenaufruf auf Referenz des Serverobjekts
MyRemoteObject.myRemotheMethod();
```

ASYNCHRONE AUFRUFE MIT RMI

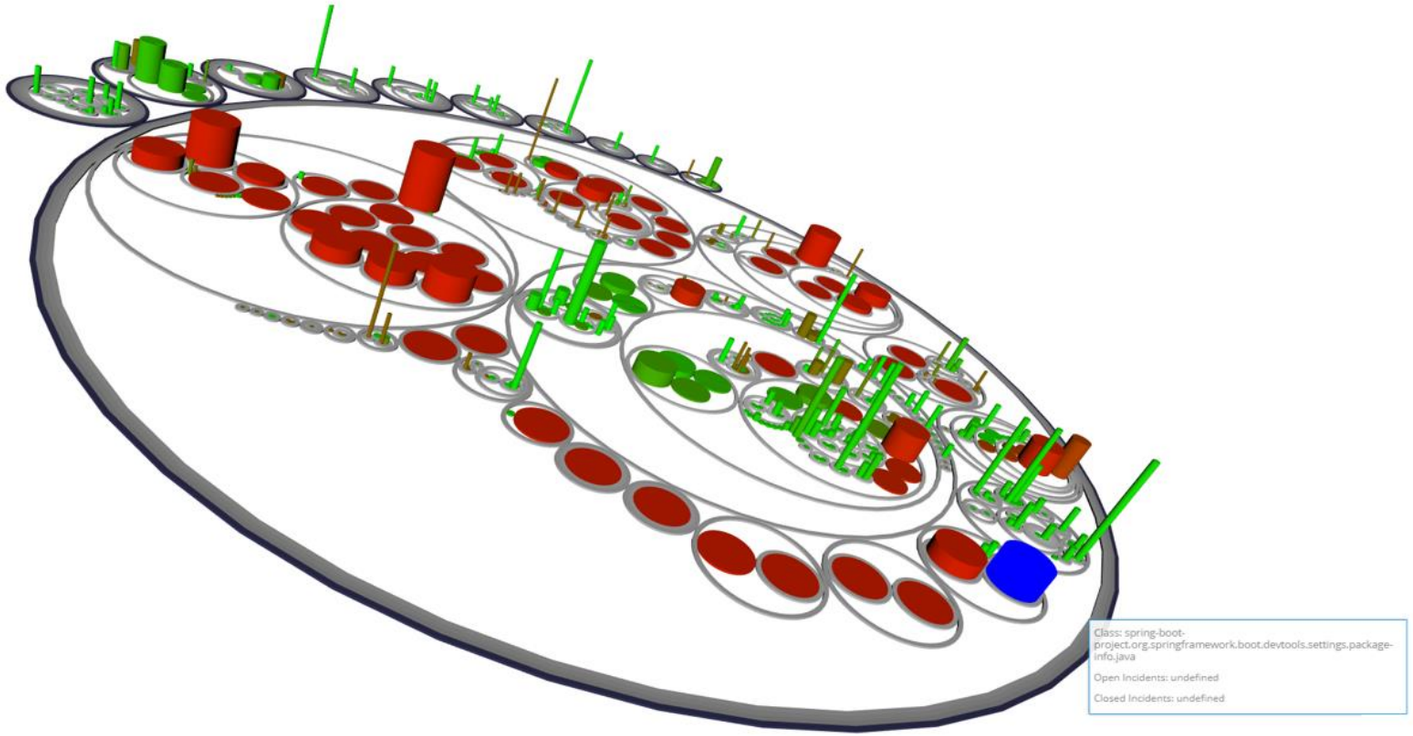
- Nicht von Haus aus unterstützt
- Main Feature ist der entfernte Methodenaufruf analog einem lokalen Aufruf, welcher per se synchron ist
- Asynchrone Kommunikation muss im Bedarfsfall selbst implementiert werden
  - Methodenaufruf per RMI auf Server
  - Servermethode startet eigenen Thread zur Verarbeitung der Anfrage
  - Thread ruft am Ende der Bearbeitung Methode auf Client per RMI
  - Clientmethode startet eigenen Thread zur Verarbeitung per Anfrage
- Alternativ asynchrone nachrichtenbasierte Kommunikation nutzen

VERTEILTE GARBAGE COLLECTION

- Java RMI unterstützt verteilte Garbage Collection
- Der Garbage-Collection-Algorithmus basiert auf Reference Counting
- Objektfreigabe bei fehlenden entfernten und lokalen Referenzen
- Client sendet regelmäßig eine Nachricht an den Server, wenn er noch mindestens eine Referenz auf das entfernte Objekt hält

UNIVERSITÄT
LEIPZIG

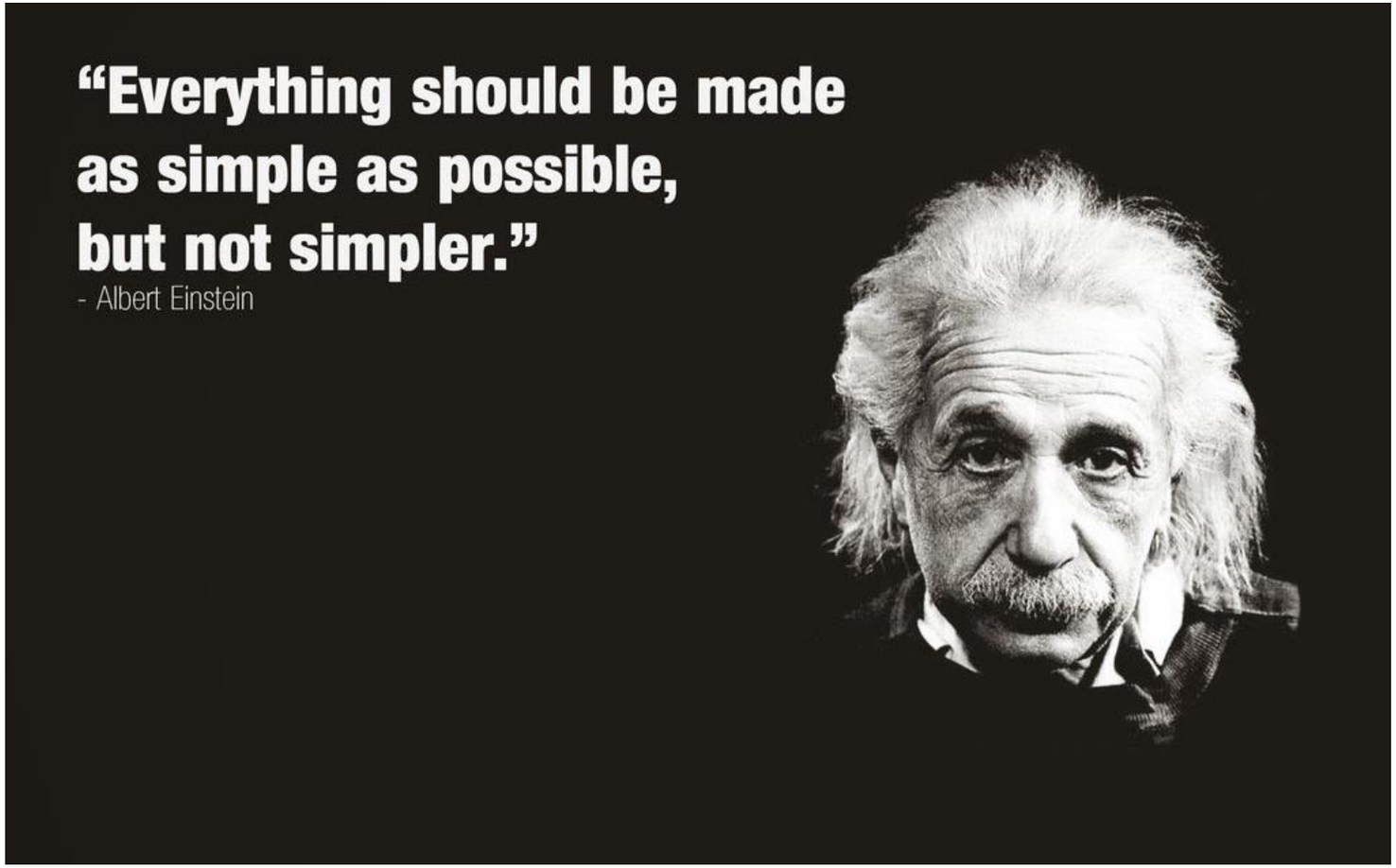
VIELEN DANK!



Dr. Pascal Kovacs

Institut für Wirtschaftsinformatik

pkovacs@uni-leipzig.de

https://www.wifa.uni-leipzig.de/iwi/

VIELEN DANK FÜR IHRE AUFMERKSAMKEIT!